INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE OCCIDENTE

Documentación Proyecto Final

Rico Quiz Plus Client

Tecnologías de Desarrollo en el Cliente

Presenta:

Andres Juarez

Adan Juarez Jr

Índice del Documento

Definición del Proyecto **4**
1. Nombre del proyecto 4
2. Integrantes 4
3. Descripción 4
4. Stack de tecnologías a utilizar 5
5. Alternativas / proyectos similares o referencias 5

Planeación y Prototipado del Proyecto Integrador **7**
Generales 7
Alcance 7
Funcionalidades incluidas 7
Fuera de alcance 7
Supuestos 8
Limitaciones identificadas 8
Riesgos identificados 8
Integraciones a terceros 8
Arquitectura 9
Stack de tecnologías 9
Flujo general 10
Diseño 10
Estructura de la base de datos 10
Autenticación 12
Roles 12
Responsabilidades por rol 12
Cronograma 13
Board de Planeación 13
Wireframes
Imágenes de referencia de la aplicación 15

Entrega 1 - Vistas del Proyecto **19**
Imágenes del Layout del Portal 20
Pantallas de Partida 36

Entrega 3 - APIs y Formularios **45**
Generales 45
Fuente de datos: API propia 45
Contrato de la Api 46
Arquitectura del Cliente 46
CRUD de Quiz 52
Formularios y validaciones 59
Validadores personalizados (src/app/shared/validators): 62
Estados de la interfaz y manejo de errores 62
Autenticación 63
Autoevaluación de la rúbrica 64

Definición del Proyecto

1. Nombre del proyecto

RicoQuiz+ (versión SPA con Angular)

2. Integrantes

- Andres
- Adán

3. Descripción

RicoQuiz+ es una plataforma de trivia multijugador en tiempo real, inspirada en experiencias tipo Kahoot / Jackbox, que ya fue desarrollada como proyecto final de la materia de Tecnologías de Desarrollo en el Servidor. En esa etapa se construyó todo el backend: un motor de juego basado en una máquina de estados sobre Socket.io (lobby → loading → question → reveal → leaderboard → finished), autenticación con JWT y activación por correo, generación de preguntas apoyada en IA (Gemini), manejo de imágenes con Cloudinary y un frontend inicial en HTML/CSS/JS y desplegado en Render.

El problema que buscamos resolver en esta segunda etapa es la limitación del frontend actual: al estar hecho con JavaScript vanilla, la interfaz es difícil de escalar y mantener, no tiene una gestión de estado clara para los distintos momentos del juego, y obliga a los usuarios a crear una cuenta con correo y contraseña incluso si solo quieren entrar rápido a jugar una partida, por los requerimientos de la clase anterior, lo cual es fricción innecesaria para un juego social pensado para usarse momentáneamente.

Para esta materia, el proyecto consiste en reconstruir y extender el cliente de RicoQuiz+ como una Single Page Application (SPA) con Angular y SASS, consumiendo la misma API REST y los mismos sockets del backend ya existente. Además de migrar la interfaz a Angular, se añadirá inicio de sesión con Google (OAuth 2.0) como alternativa al registro tradicional, para que unirse a una partida sea más rápido y natural, sin perder la opción de autenticación por correo ya construida en el backend.

En resumen, la solución propuesta combina:

- Migración del frontend de HTML/CSS/JS plano a una SPA en Angular con SASS.
- Enrutamiento del lado del cliente para las distintas vistas (inicio, sala de espera, juego, resultados, historial de partidas).

- Inicio de sesión con Google como método de autenticación adicional al ya existente.
- Reutilización íntegra del backend y del motor de juego en tiempo real ya construidos.

4. Stack de tecnologías a utilizar

Área	Tecnología / Herramienta
Frontend / SPA	Angular (última versión estable), TypeScript, SASS (SCSS) para estilos, Angular Router, Angular Reactive Forms
Comunicación en tiempo real	Socket.io-client (reutilizando el motor de juego del backend
Autenticación	Google Identity Services (OAuth 2.0) para inicio de sesión con Google, combinado con el JWT + activación por correo ya existente en el backend
Backend (heredado, sin cambios mayores)	Node.js, Express, TypeScript, MongoDB / Mongoose, Socket.io, Gemini AI (generación de preguntas), Cloudinary (imágenes), Brevo (correo)
Documentación de API	OpenAPI / Swagger (ya existente en el backend)
Control de estado en el cliente	Angular Signals para el manejo reactivo del estado del juego
Hosting / Cloud	Backend en Render (ya desplegado); frontend Angular en Vercel, Netlify o Firebase Hosting
Herramientas de build	Angular CLI, npm

5. Alternativas / proyectos similares o referencias

El proyecto está basado en la primera versión de RicoQuiz+, desarrollada por el mismo equipo como proyecto final de Tecnologías de Desarrollo en el Servidor, y en su predecesor RicoQuiz (versión con WebSockets planos). Se decidió continuar y extender ese proyecto en lugar de partir de cero porque ya se cuenta con un backend funcional, probado y desplegado, y porque esta materia da la oportunidad de resolver justamente la parte que quedó pendiente: un frontend completo, mantenible y con un flujo de acceso más simple para los jugadores.

Como referencias de la industria para este tipo de plataformas de trivia/juegos en vivo se consideraron:

- Kahoot! - plataforma educativa de trivia en vivo, referencia principal en dinámica de juego (pines de sala, ranking en tiempo real).
- Jackbox Games - juegos de fiesta multijugador que se controlan desde el celular mientras se ve una pantalla compartida.
- Quizizz - variante asíncrona de trivia con enfoque educativo y reportes de desempeño.
- Blooket - trivia gamificada con mecánicas adicionales tipo coleccionables.

A diferencia de estas plataformas, RicoQuiz+ se desarrolla como proyecto propio con fines de aprendizaje, permitiendo control total sobre el motor de juego, y principalmente la posibilidad de crear quizzes o partidas basadas en un tema y generado con IA.

Planeación y Prototipado del Proyecto Integrador

Generales

Nombre del proyecto:

RicoQuiz+ (versión SPA con Angular)

Integrantes:

Andrés Juárez, Adán Juárez

Descripción general:

Reconstrucción del cliente de RicoQuiz+ (trivia multijugador en tiempo real, estilo Kahoot) como una Single Page Application en Angular + SASS, reutilizando íntegramente el backend ya construido en Tecnologías de Desarrollo en el Servidor: API REST, motor de juego con Socket.io (máquina de estados lobby → loading → question → reveal → leaderboard → finished), autenticación JWT, generación de preguntas con Gemini y manejo de imágenes con Cloudinary.

Problema que resuelve:

El frontend actual está hecho en HTML/CSS/JS plano, lo que dificulta su mantenimiento y no tiene una gestión de estado clara para los distintos momentos del juego. Además, exige crear una cuenta con correo y contraseña incluso para unirse rápido a una partida, lo cual es fricción innecesaria en un juego social pensado para sesiones cortas. Esta etapa migra la interfaz a Angular y añade login con Google como alternativa más rápida al registro tradicional.

Alcance

Funcionalidades incluidas

- Migración completa de la interfaz de HTML/CSS/JS plano a una SPA en Angular con SASS.
- Enrutamiento del lado del cliente con Angular Router para las vistas: inicio, sala de espera, juego, resultados e historial de partidas.
- Consumo de la API REST y de los sockets ya existentes del backend, sin alterar sus contratos.
- Inicio de sesión con Google (OAuth 2.0) como alternativa al registro por correo/contraseña ya existente.
- Manejo reactivo del estado del juego en el cliente con Angular Signals (sincronizado con la máquina de estados del backend).

Fuera de alcance

- Cambios al motor de juego, al modelo de datos o a los endpoints del backend más allá de lo estrictamente necesario para servirlos desde Angular.
- Monetización o pagos.

- Aplicación móvil nativa (solo web responsiva).
- Nuevos tipos de pregunta o mecánicas de juego distintas a las ya soportadas (opción múltiple).

Supuestos

- El backend desplegado en Render se mantiene disponible y estable durante el semestre.
- Las credenciales de Gemini, Cloudinary y Brevo ya configuradas siguen vigentes.
- Se cuenta con (o se da de alta) un Client ID de Google OAuth propio para este frontend.

Limitaciones identificadas

- Equipo de 2 personas con el tiempo de un semestre, en paralelo a otras materias.
- El contrato de la API REST y de los eventos de socket ya están definidos desde el semestre anterior, sin embargo es posible su ajuste.

Riesgos identificados

Riesgo	Impacto	Mitigación
Desincronización entre el estado del juego en Angular Signals y los eventos de Socket.io	Alto: rompe la experiencia en vivo	Centralizar la escucha de sockets en un único servicio y derivar todo el estado de UI desde ahí
Redirect URI de Google OAuth distinto entre localhost y producción	Medio: bloquea el login en despliegue	Configurar ambos orígenes autorizados desde el inicio en Google Cloud Console
Curva de aprendizaje de Angular (equipo viene de JS/Node plano)	Medio: retrasa fases iniciales	Empezar por las vistas sin socket (inicio, historial) antes que la vista de juego en vivo
Cuota/límite de la API de Gemini para generación de preguntas	Bajo: ya mitigado en el backend existente	Ninguna acción nueva, es un riesgo heredado del proyecto base

Integraciones a terceros

- **Google Identity Services (OAuth 2.0)** nueva integración en el frontend para login alterno
- **Gemini AI,** ya integrada en el backend, se reutiliza sin cambios.
- **Cloudinary**, ya integrada en el backend, se reutiliza sin cambios.
- **Brevo (correo)**, ya integrada en el backend, se reutiliza sin cambios.

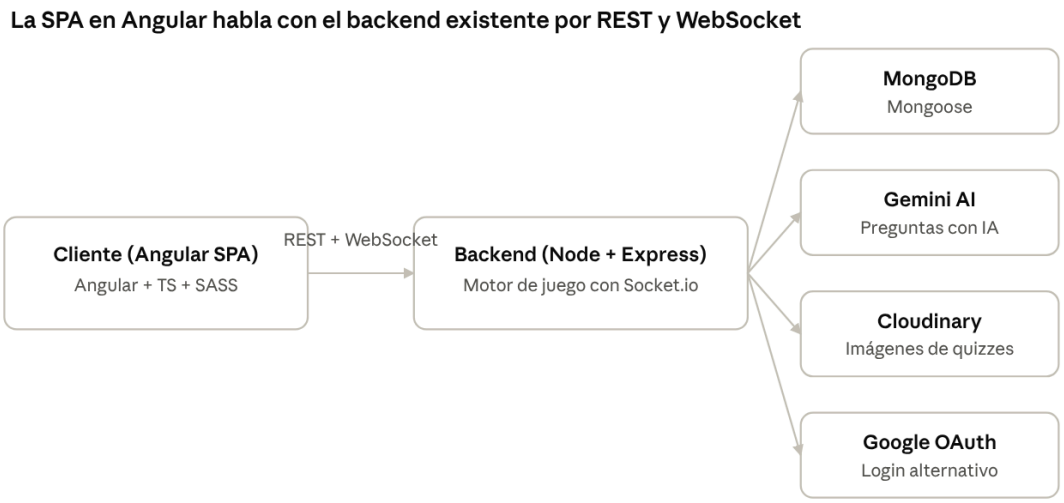
Arquitectura

Tipo de aplicación:

Cliente-servidor. Un único backend (Node/Express + Socket.io) sirve tanto la API REST como el canal en tiempo real; el frontend es una SPA en Angular que consume ambos. No hay microfrontends ni backend-for-frontend adicional.

Stack de tecnologías

Área	Tecnología
Frontend / SPA	Angular (última estable), TypeScript, SASS, Angular Router, Angular Reactive Forms
Comunicación en tiempo real	Socket.io-client, consumiendo el motor de juego existente
Autenticación	Google Identity Services (OAuth 2.0) + JWT/activación por correo ya existente
Backend (heredado)	Node.js, Express, TypeScript, MongoDB/Mongoose, Socket.io, Gemini AI, Cloudinary, Brevo
Documentación de API	OpenAPI/Swagger (ya existente)
Estado del cliente	Angular Signals
Hosting	Backend en Render (ya desplegado); frontend en Vercel, Netlify o Firebase Hosting
Build	Angular CLI, npm



arquitectura cliente-servidor · SPA Angular + backend existente

Flujo general

1. El jugador abre la SPA, inicia sesión con Google o con correo/JWT, o entra directo con un código de sala como invitado.
2. La SPA pide por REST el quiz y crea/consulta la sala; a partir de ahí todo el juego se coordina por Socket.io.
3. El backend conduce la partida por su máquina de estados (`lobby → loading → question → reveal → leaderboard → finished`), emitiendo eventos que la SPA refleja en Angular Signals.
4. Al terminar, el resultado queda disponible por REST (`GET /matches/:id`, `GET /users/:id/matches`) para el historial.

Diseño

Estructura de la base de datos

Tres colecciones en MongoDB, heredadas sin cambios del backend (Mongoose, con `timestamps: true` en las tres):

User

Campo	Tipo	Notas
_id	ObjectId	
email	String	único
displayName	String	
avatarUrl	String	
googleId	String	presente si inició sesión con Google

Campo	Tipo	Notas
passwordHash	String	presente si se registró por correo
createdQuizzes[]	[ObjectId]	denormalizado, referencia a Quiz

Quiz

Campo	Tipo	Notas
_id	ObjectId	
title	String	
topic	String	
tags[]	[String]	
isPublic	Boolean	
owner	ObjectId → User	
questions[]	[{question, options[], correctIndex}]	

Match

Campo	Tipo	Notas
_id	ObjectId	
roomCode	String	código de sala
quiz	ObjectId → Quiz	
host	ObjectId → User	
players[]	[{user, score}]	user puede ser invitado sin cuenta
status	enum: waiting | in_progress | finished	

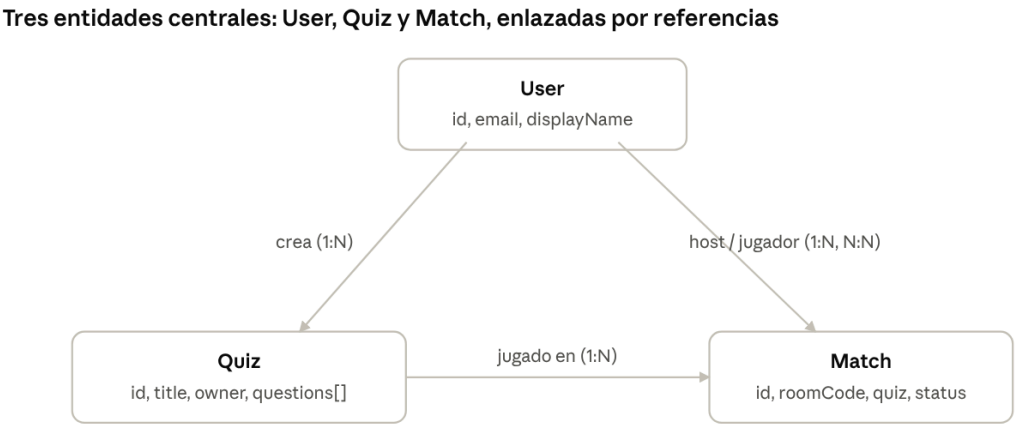


diagrama entidad-relación · User, Quiz, Match

Autenticación

Se mantiene el esquema JWT ya existente (registro por correo + activación) y se añade Google OAuth 2.0 (Google Identity Services) como segundo método de entrada. Ambos flujos terminan emitiendo el mismo JWT, así que el resto de la aplicación, REST y sockets, no distingue cómo inició sesión el usuario: el token viaja en el header `Authorization` para REST y se valida al conectar el socket.

Se conserva también la entrada como invitado por código de sala, sin cuenta, para no forzar login en partidas casuales.

Roles

Rol	Puede
Invitado (sin cuenta)	Unirse a una partida con código, jugar, ver resultados de esa partida
Usuario autenticado (correo o Google)	Todo lo anterior, además de crear/administrar sus propios quizzes, iniciar partidas como host y ver su historial de partidas

Responsabilidades por rol

- **Invitado:** responde preguntas y aparece en el leaderboard de la partida en curso; no persiste historial entre sesiones.
- **Usuario autenticado:** dueño de sus quizzes (`owner` en Quiz) y, cuando actúa como host de una partida, controla el avance de la máquina de estados del juego (iniciar, avanzar de pregunta); como jugador tiene los mismos permisos que un invitado más el historial persistente.

Cronograma

Fase	Duración estimada	Entregable principal
1. Planeación y diseño	1 semana	Este documento, board en GitHub Projects, wireframes
2. Setup Angular + routing base	1 semana	Esqueleto de la SPA con las rutas y el layout principal
3. Integración REST (quizzes, usuarios)	1 semana	Vistas de inicio, exploración de quizzes e historial conectadas a la API real
4. Integración Socket.io (juego en vivo)	2 semanas	Lobby, pantalla de juego y leaderboard funcionando de punta a punta
5. Google OAuth	1 semana	Login con Google funcional junto al login por correo existente
6. Pulido, pruebas y despliegue	1 semana	Frontend desplegado (Vercel/Netlify/Firebase Hosting) y QA final

Board de Planeación

Para la organización y seguimiento del proyecto se utilizará GitHub Projects, donde se concentrarán las actividades relacionadas con el desarrollo del proyecto. El tablero permitirá visualizar el progreso, organizar el backlog y dar seguimiento a las tareas conforme avance el proyecto.

Enlace al tablero de GitHub: https://github.com/users/andresjuar/projects/3

Las actividades se encuentran organizadas mediante ocho épicas, que agrupan tareas relacionadas con una misma área funcional o técnica del proyecto:

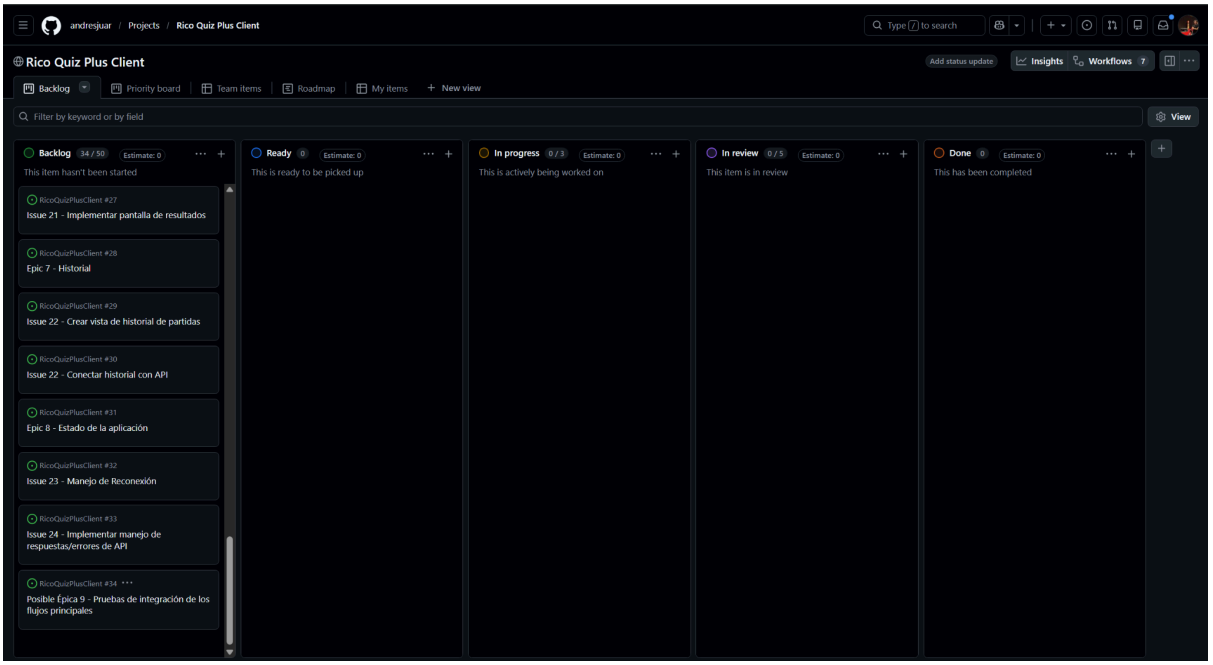
- **Configuración y arquitectura Angular:** configuración inicial del proyecto, estructura de la aplicación, routing y herramientas necesarias para el desarrollo.
- **Autenticación:** inicio de sesión, registro, manejo de sesión, JWT, autorización y autenticación mediante Google.
- **Inicio y navegación:** implementación de las vistas principales y navegación entre las diferentes secciones de la aplicación.

- **Creación y administración de quizzes:** formularios, configuración y comunicación con la API para la creación y gestión de quizzes.
- **Lobby y conexión a partidas:** incorporación de jugadores, manejo de salas y comunicación mediante Socket.io.
- **Juego en tiempo real:** preguntas, respuestas, temporizadores, resultados y leaderboard.
- **Historial y resultados:** consulta y presentación de partidas y resultados anteriores.
- **Estado de la aplicación:** manejo del estado de autenticación y partida mediante Angular Signals, incluyendo situaciones de desconexión y reconexión.
- **Pruebas de integración de los flujos principales:** estas actividades serán para verificar que el desarrollo fue correcto, actualmente no hay actividades designadas para esta épica pero serán descritas conforme avance el desarrollo del proyecto.

Las actividades serán divididas y asignadas progresivamente conforme avance el desarrollo, considerando las necesidades y prioridades de cada etapa del proyecto. La asignación no será necesariamente fija desde el inicio, ya que podrá modificarse conforme aparezcan nuevas necesidades, bloqueos o cambios en las actividades.

El objetivo será mantener una distribución de trabajo lo más equitativa posible entre los integrantes del equipo, procurando que todos participen en diferentes áreas del proyecto y que la carga de trabajo sea razonablemente equilibrada. Para facilitar este seguimiento, cada actividad contará con un responsable y una estimación de esfuerzo.

El board utilizará estados que permitan visualizar el progreso de las actividades, desde su identificación y preparación hasta su finalización, facilitando la coordinación del equipo y el seguimiento del avance general del proyecto, los estados son: Backlog, Ready, In Progress, In review y Done.



Wireframes

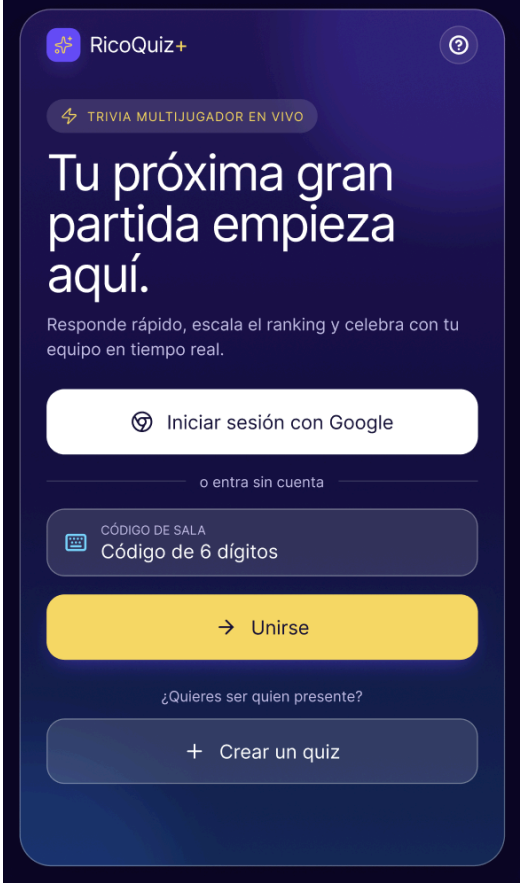
Imágenes de referencia de la aplicación

https://www.figma.com/design/opMrxAEjP9T3Fd2eBvYoCy/Untitled?node-id=0-1&t=FSn7fmwWw6Jw7av6-1

El enlace no se pudo compartir directamente, requiere otorgar permisos, para evitar eso, a continuación se muestran capturas de las pantallas:

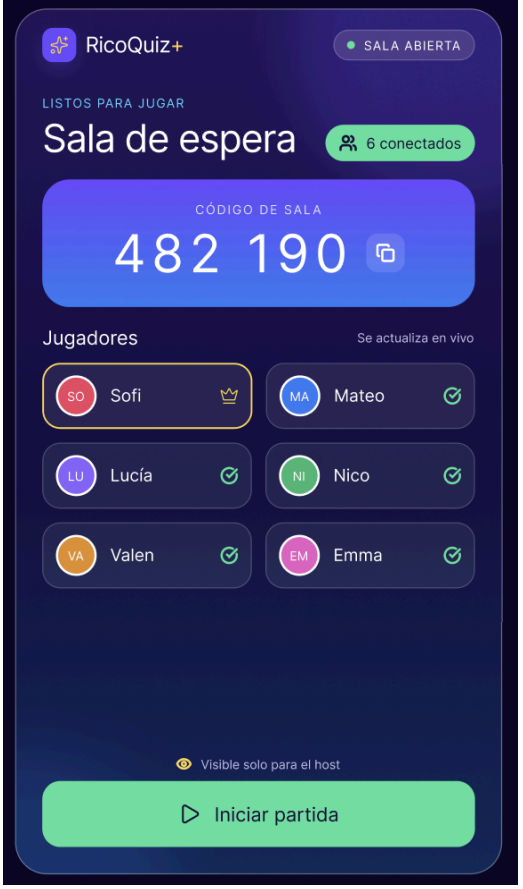
01 — ENTRADA

Inicio



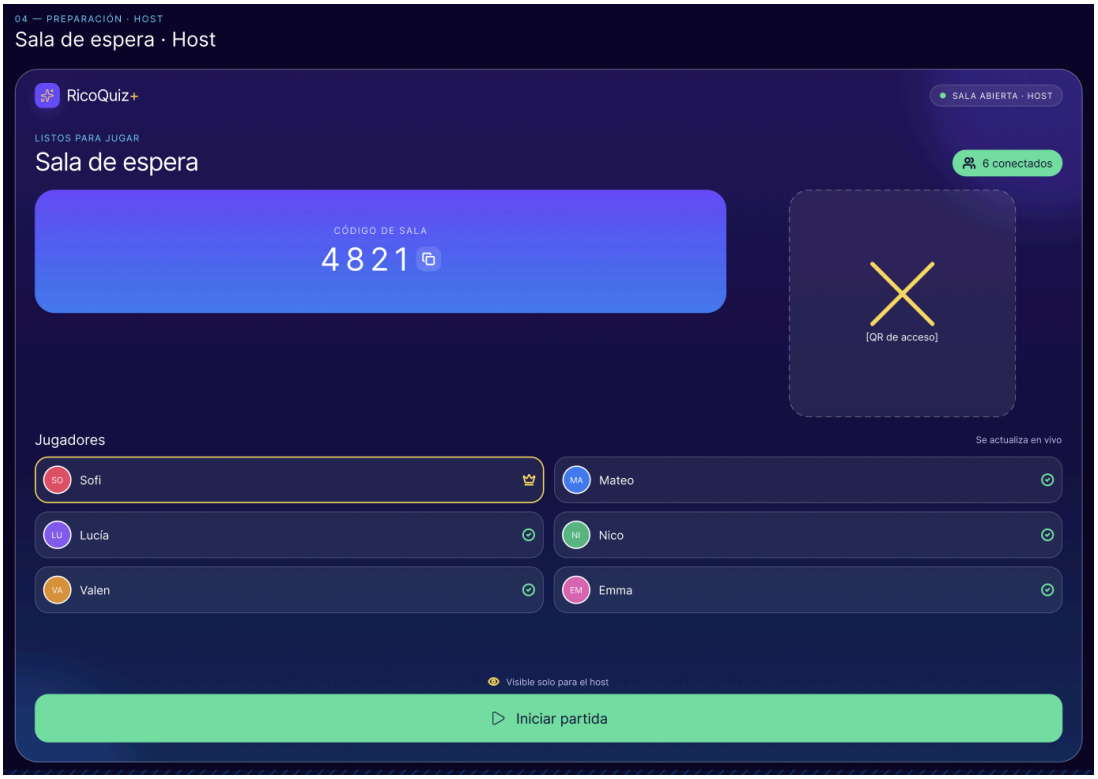
02 — PREPARACIÓN

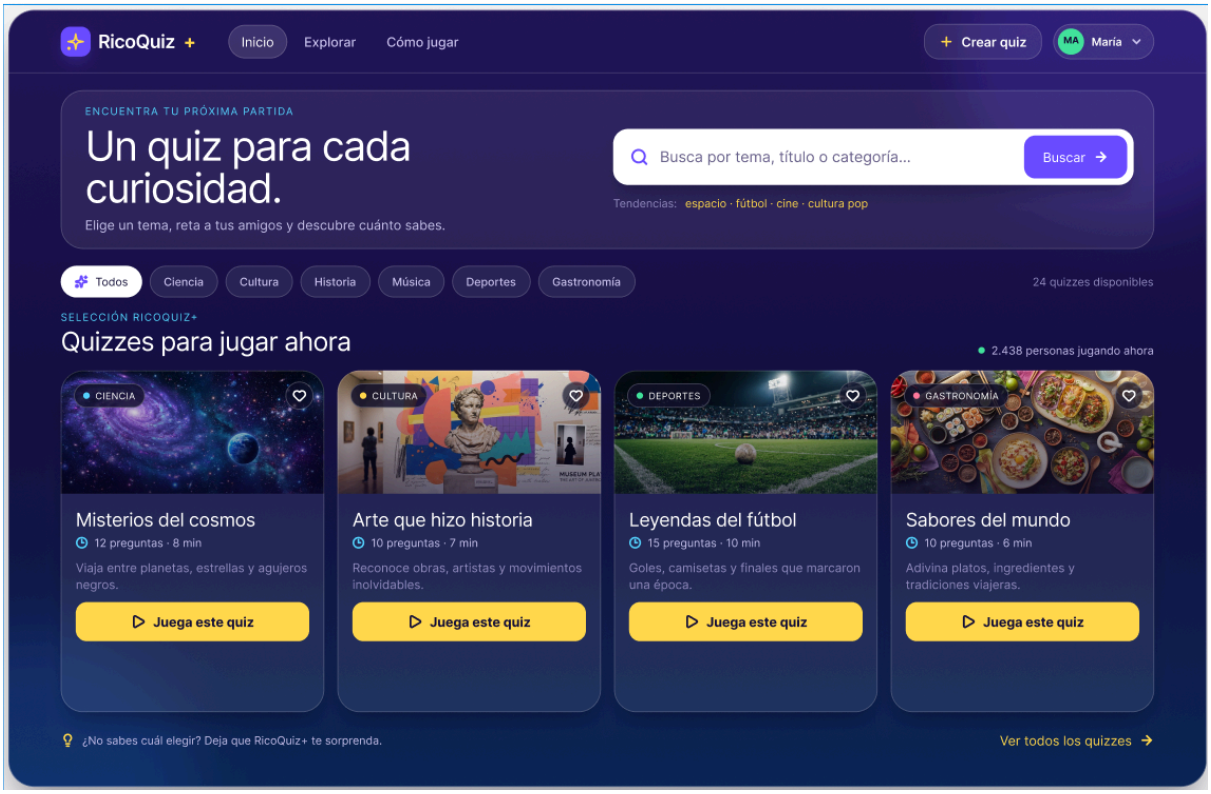
Sala de espera host



03 — EN DIRECTO
Juego
Se repite por cada pregunta
RicoQuiz+
EN JUEGO
Pregunta 3
de 10
18
SEG
CIENCIA · PLANETA TIERRA
¿Cuál es el océano más grande del planeta?
Atlántico
Pacífico
Índico
Ártico
7 de 8 respondieron
88%

05 — CIERRE
Resultados
RicoQuiz+
FINAL
RESULTADOS FINALES
¡Partida terminada!
#2
MA
Mateo
8.760 pts
#1
SO
Sofi
9.420 pts
#3
LU
Lucía
7.980 pts
Clasificación
8 jugadores
4 NI Nico 7.420 pts
5 VA Valen 6.890 pts
6 EM Emma 6.310 pts
Jugar de nuevo
Salir


04 — PREPARACIÓN · HOST
Sala de espera · Host
RicoQuiz+
SALA ABIERTA · HOST
LISTOS PARA JUGAR
Sala de espera
6 conectados
CÓDIGO DE SALA
4821
[QR de acceso]
Jugadores
Se actualiza en vivo
Sofi
Mateo
Lucía
Nico
Valen
Emma
Visible solo para el host
Iniciar partida


RicoQuiz +
Inicio
Explorar
Cómo jugar
+ Crear quiz
MA
María
ENCUENTRA TU PRÓXIMA PARTIDA
Un quiz para cada curiosidad.
Elige un tema, reta a tus amigos y descubre cuánto sabes.
Busca por tema, título o categoría...
Buscar
Tendencias: espacio · fútbol · cine · cultura pop
Todos
Ciencia
Cultura
Historia
Música
Deportes
Gastronomía
24 quizzes disponibles
SELECCIÓN RICOQUIZ+
Quizzes para jugar ahora
2.438 personas jugando ahora
CIENCIA
Misterios del cosmos
12 preguntas · 8 min
Viaja entre planetas, estrellas y agujeros negros.
Juega este quiz
CULTURA
Arte que hizo historia
10 preguntas · 7 min
Reconoce obras, artistas y movimientos inolvidables.
Juega este quiz
DEPORTES
Leyendas del fútbol
15 preguntas · 10 min
Goles, camisetas y finales que marcaron una época.
Juega este quiz
GASTRONOMÍA
Sabores del mundo
10 preguntas · 6 min
Adivina platos, ingredientes y tradiciones viajeras.
Juega este quiz
¿No sabes cuál elegir? Deja que RicoQuiz+ te sorprenda.
Ver todos los quizzes

Entrega 1 - Vistas del Proyecto

RicoQuiz+ es una Single Page Application en Angular 22. Todas las páginas se cargan bajo demanda (*lazy loading*) y la navegación ocurre sin recargar el navegador. La aplicación tiene dos tipos de layout. El portal (inicio, autenticación, quizzes, historial) comparte una barra de navegación superior. La partida ocupa la pantalla completa para que nada distraiga durante el juego.

En esta etapa las vistas trabajan con datos simulados: los servicios devuelven datos estáticos con un pequeño retraso que imita la latencia de red, sin hacer peticiones al API.

URL del repositorio: https://github.com/andresjuar/RicoQuizPlusClient

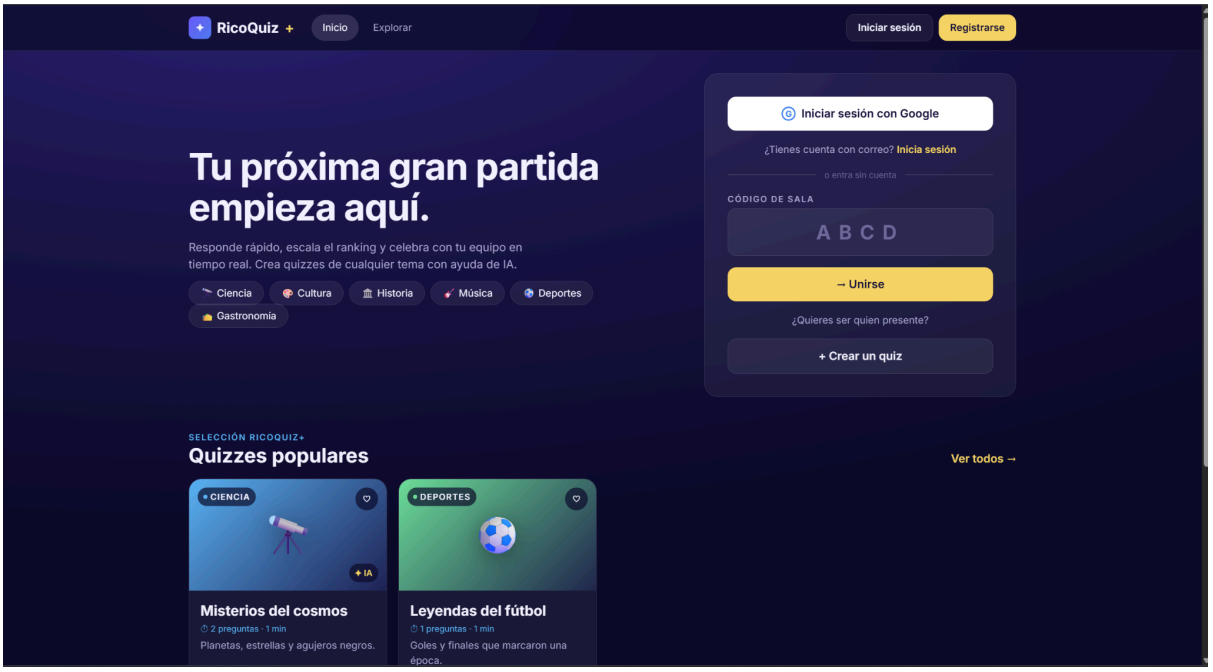
URL del Proyecto: https://github.com/users/andresjuar/projects/3

Ruta	Pantalla	Acceso
/	Inicio	Público
/explore	Explorar quizzes	Público
/login, /register	Autenticación	Solo sin sesión
/activate/:token	Activación de cuenta	Público
/dashboard	Panel del usuario	Requiere sesión
/quizzes/mine	Mis quizzes	Requiere sesión
/quizzes/new, /quizzes/:id/edit	Crear / editar quiz	Requiere sesión
/history	Historial de partidas	Requiere sesión
/room/new	Configurar nueva partida	Requiere sesión

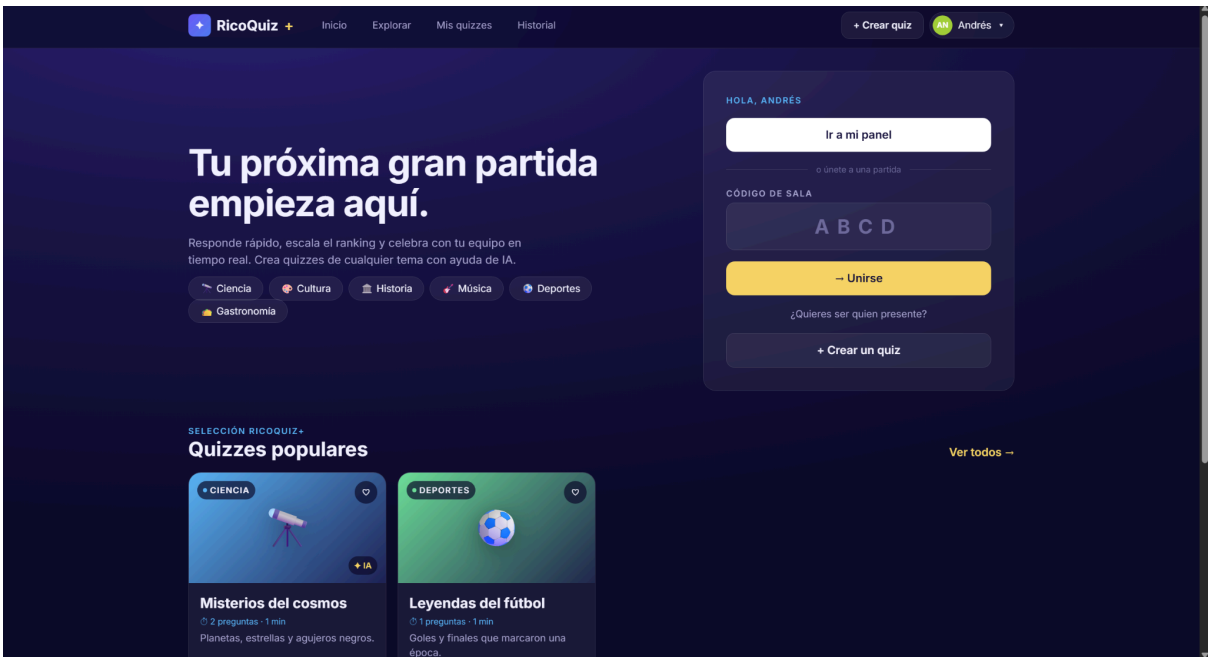
/play/:code	Partida en tiempo real	Público (invitados incluidos)
**	Página no encontrada	Público

Imágenes del Layout del Portal

La barra de navegación cambia según el estado de sesión. Un visitante ve los enlaces de Inicio y Explorar, más los botones de iniciar sesión y registrarse. Un usuario autenticado ve además “Mis quizzes”, “Historial”, el botón “Crear quiz” y un menú con su avatar para crear una partida o cerrar sesión. En pantallas pequeñas, los enlaces se agrupan en un menú desplegable.

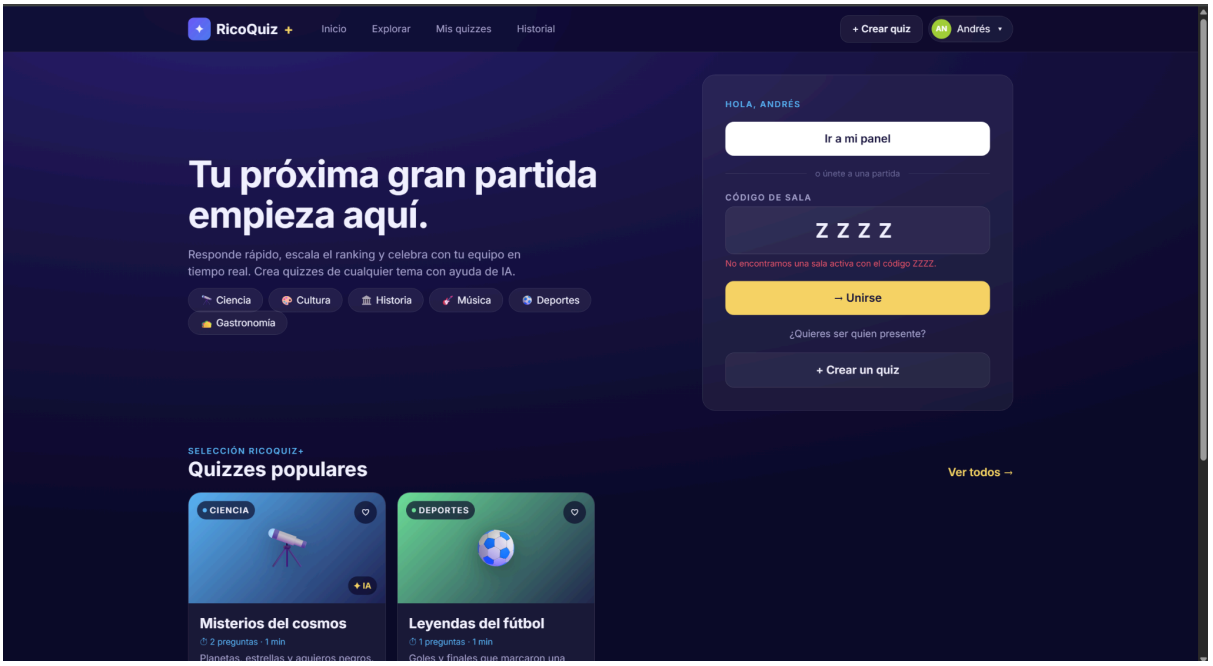


Página de Inicio sin sesión iniciada

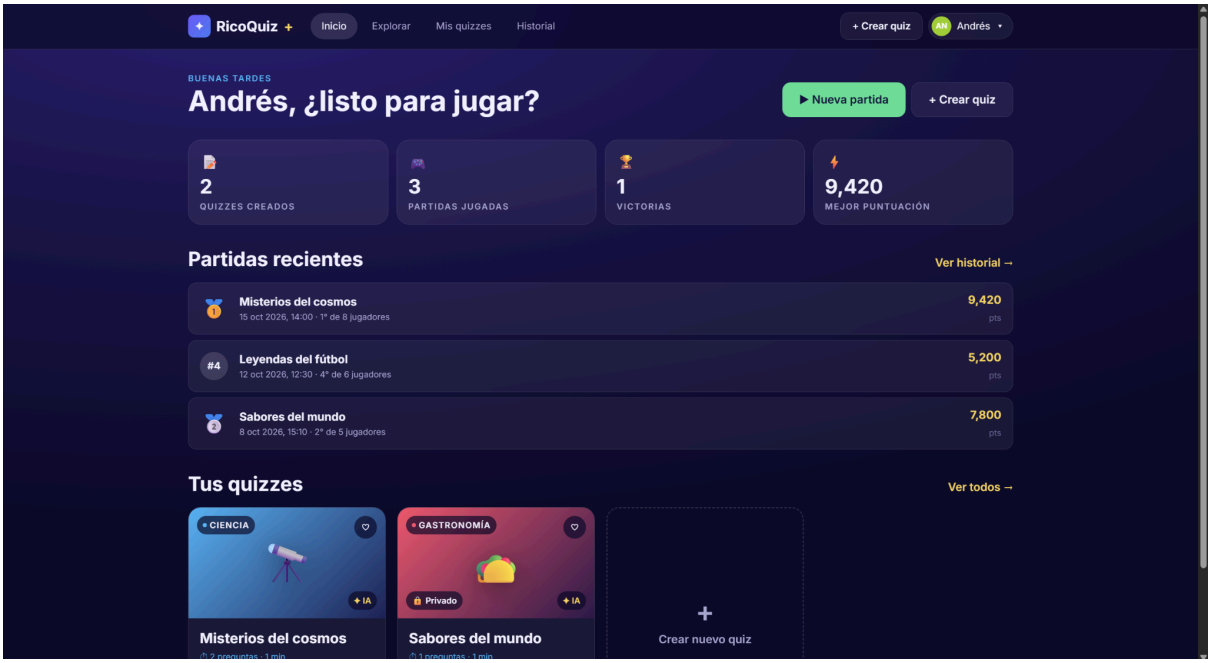


Página de inicio con sesión iniciada (se añaden rutas en la navbar)

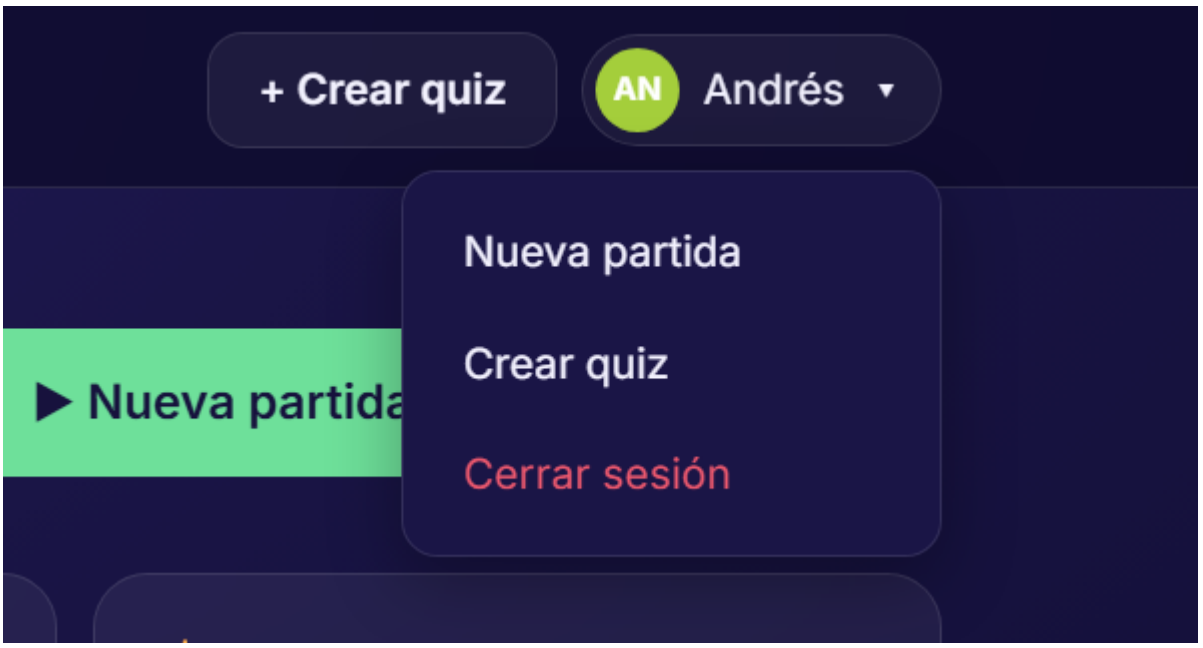
Se implementaron Guardias de inicio de sesión, si un usuario sin sesión intenta ingresar a una página que la requiere, lo envía al log in, pero una vez iniciado lo devuelve a la página que buscaba. Por el caso contrario, si un usuario logueado intenta ir a log in, lo devuelve a la pantalla de dashboard.

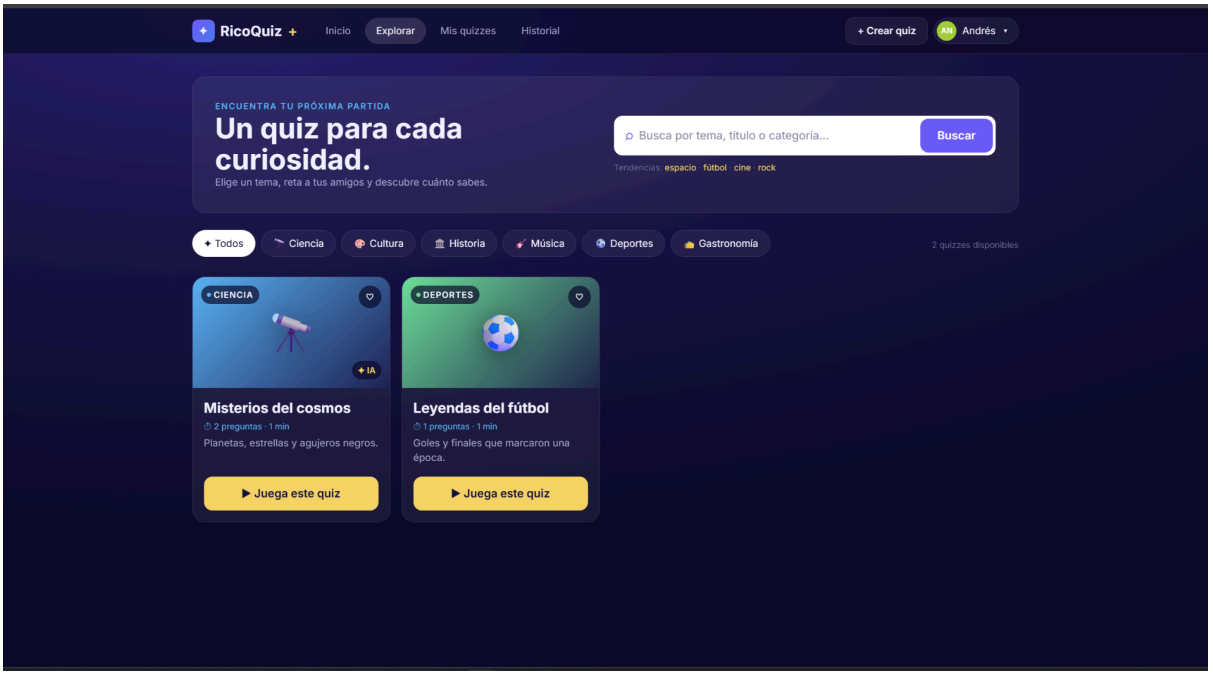


Estado de error de un código incorrecto

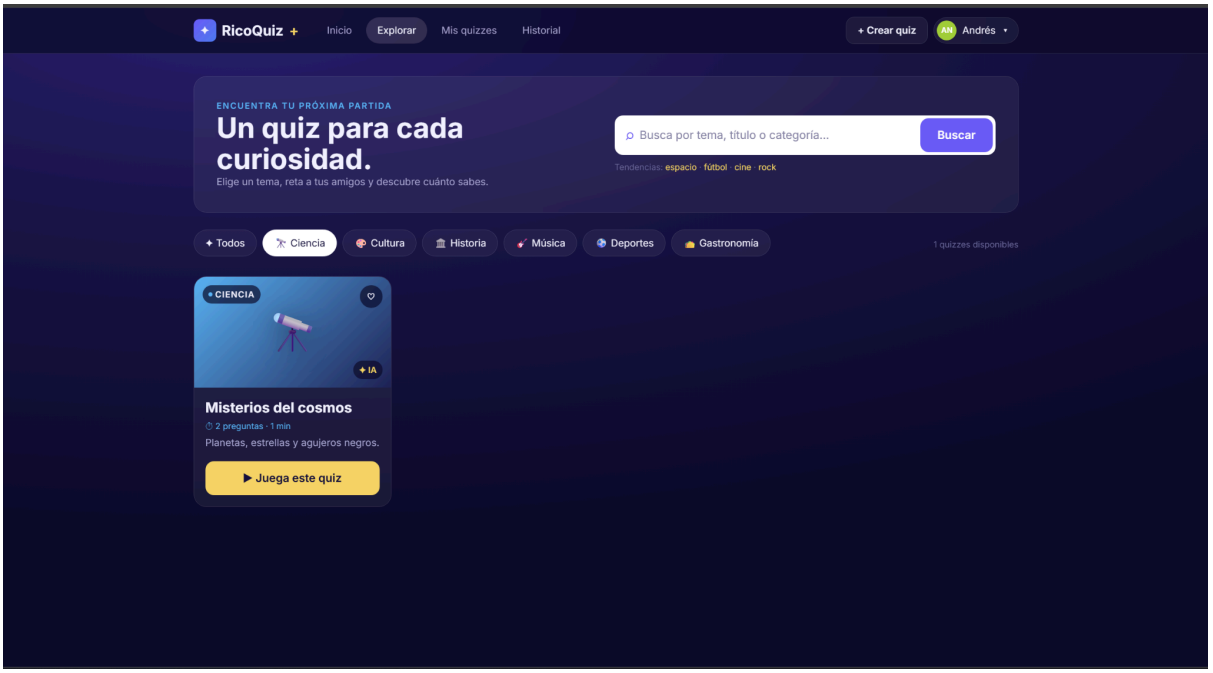


Pantalla de inicio o dashboard que solo se accede con sesión iniciada.

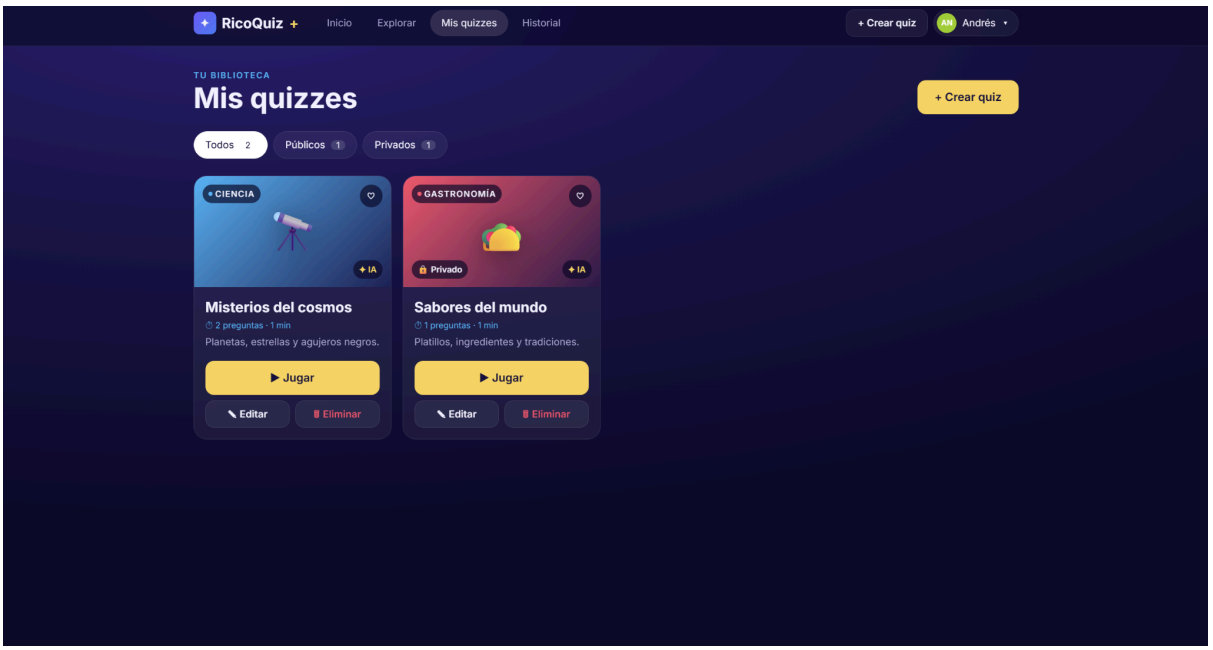




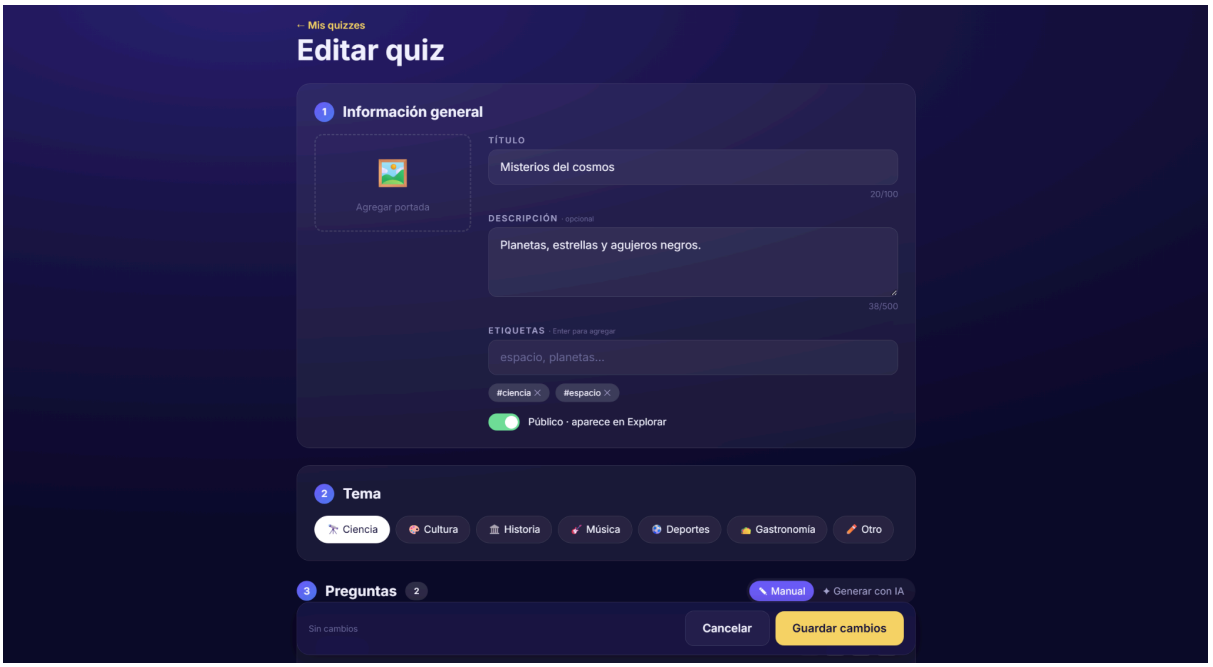
Pantalla de explorar, que tiene una barra de búsqueda y los filtros funcionan, si se le da click a una etiqueta desde la pantalla de inicio se redirige a esta pantalla con la etiqueta seleccionada.



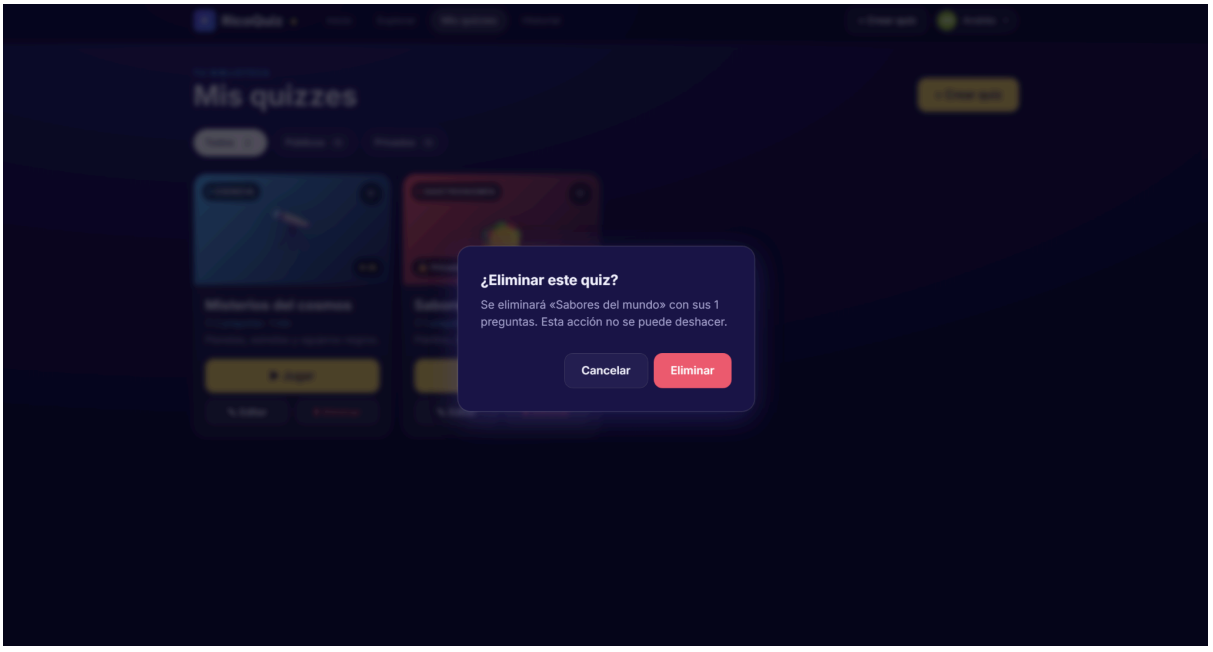
Pantalla de mis quizzes con filtros y botón de crear quiz:



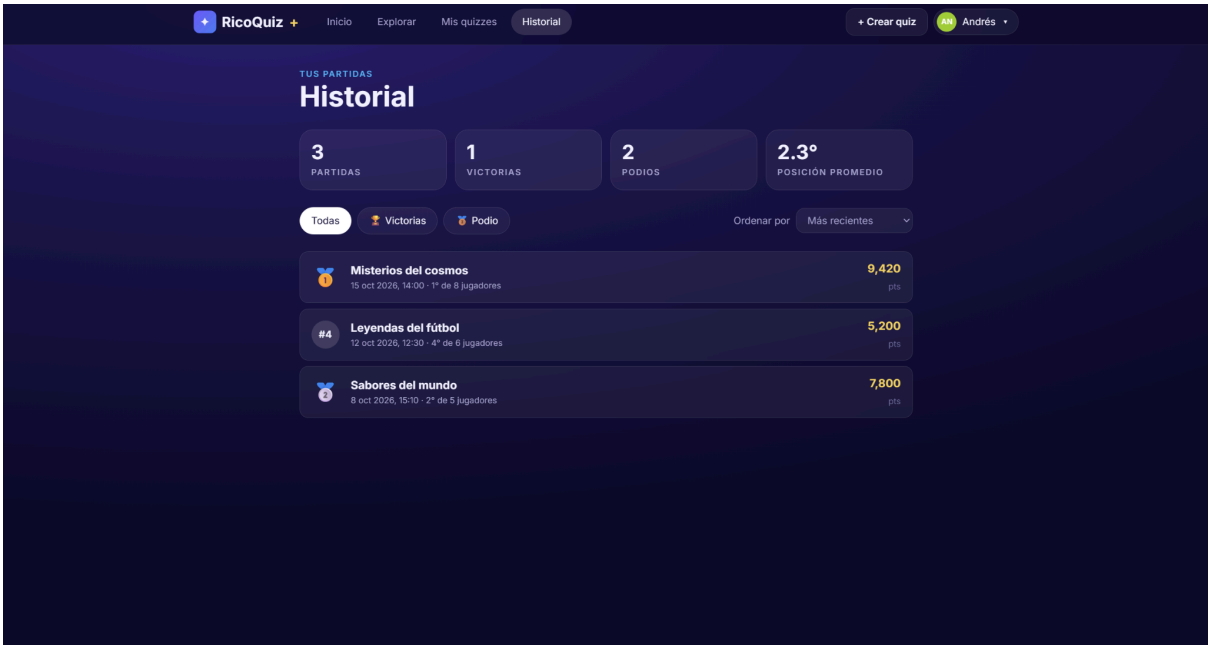
Editar un quiz ya creado:



Confirmación de eliminación de un quiz:

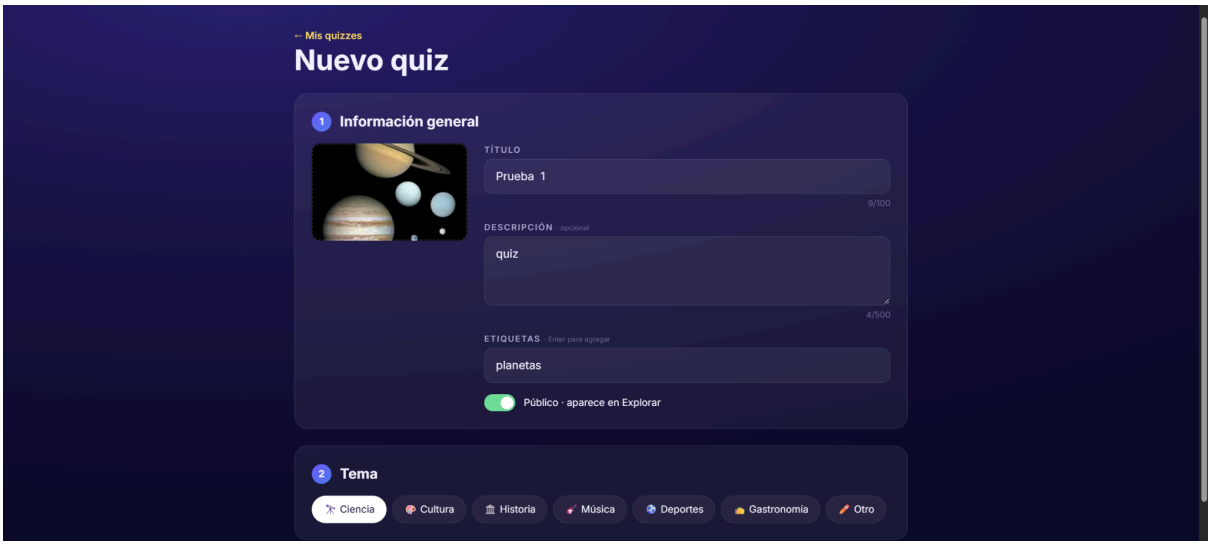


Historial de partidas jugadas:

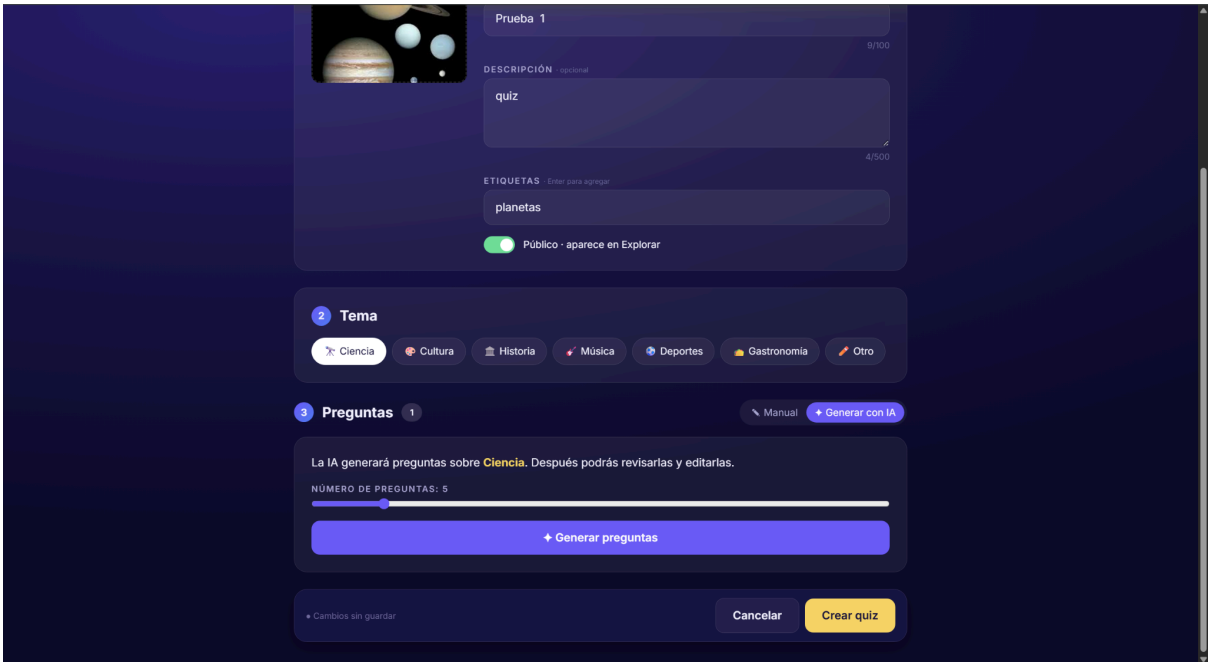


Pantallas de crear quiz:

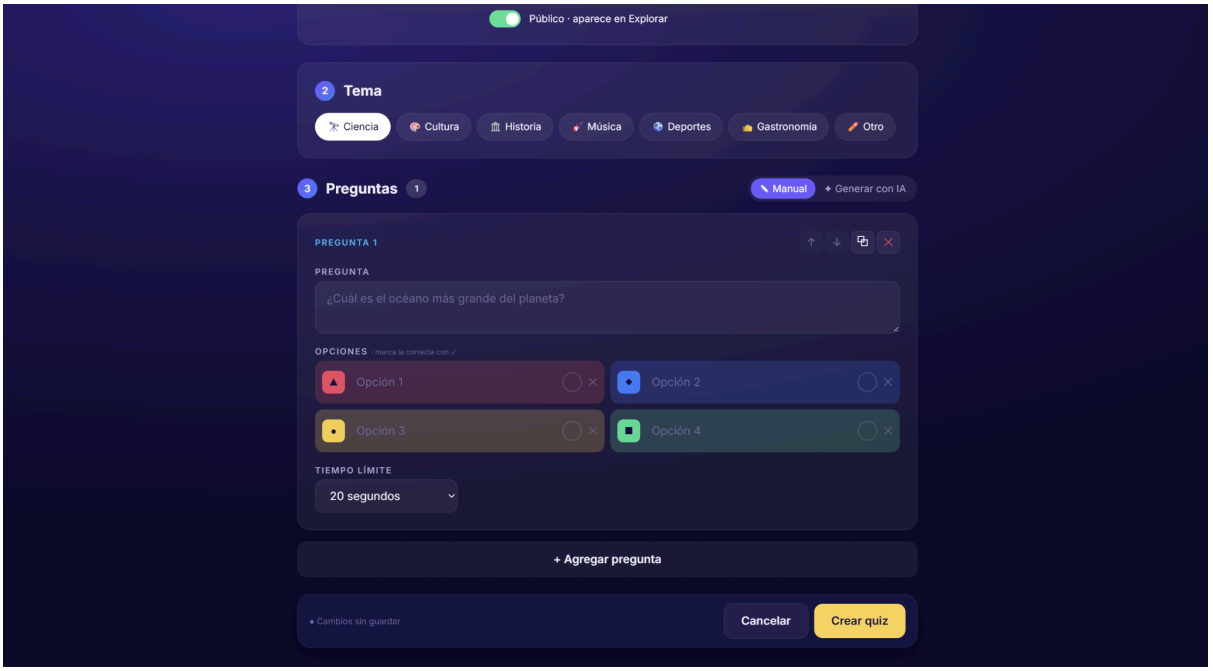
Se puede llenar el formulario



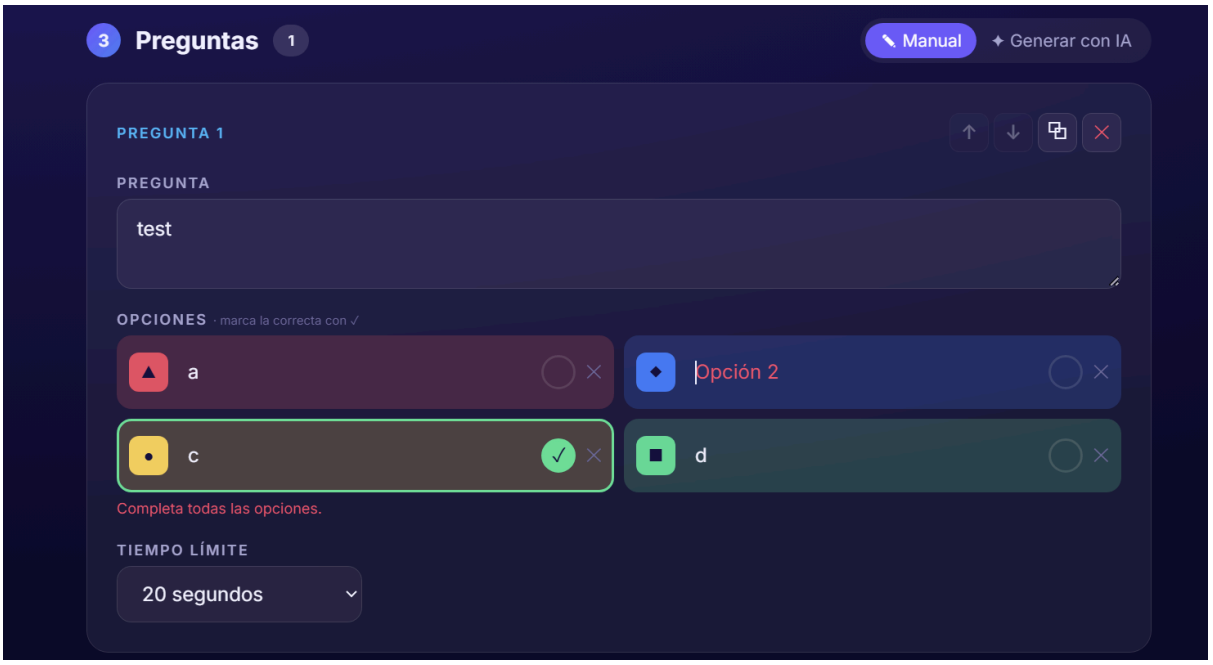
Se pueden generar preguntas con IA y después se editarían



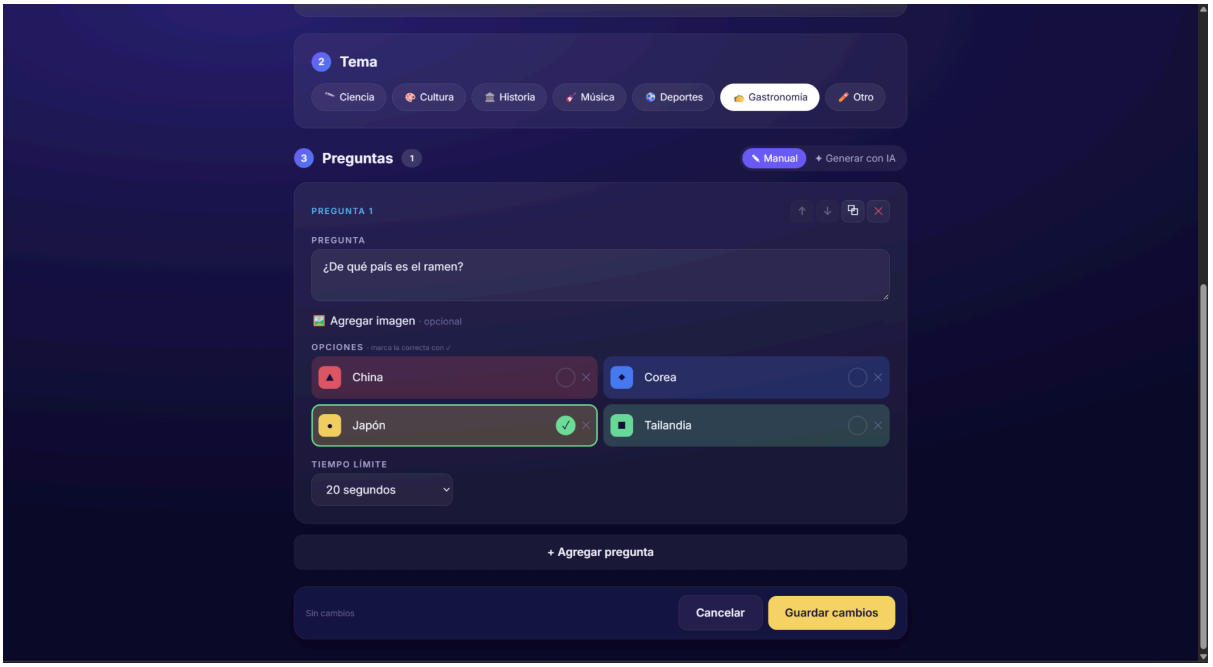
O manual y se pueden agregar de 2 a 4 opciones



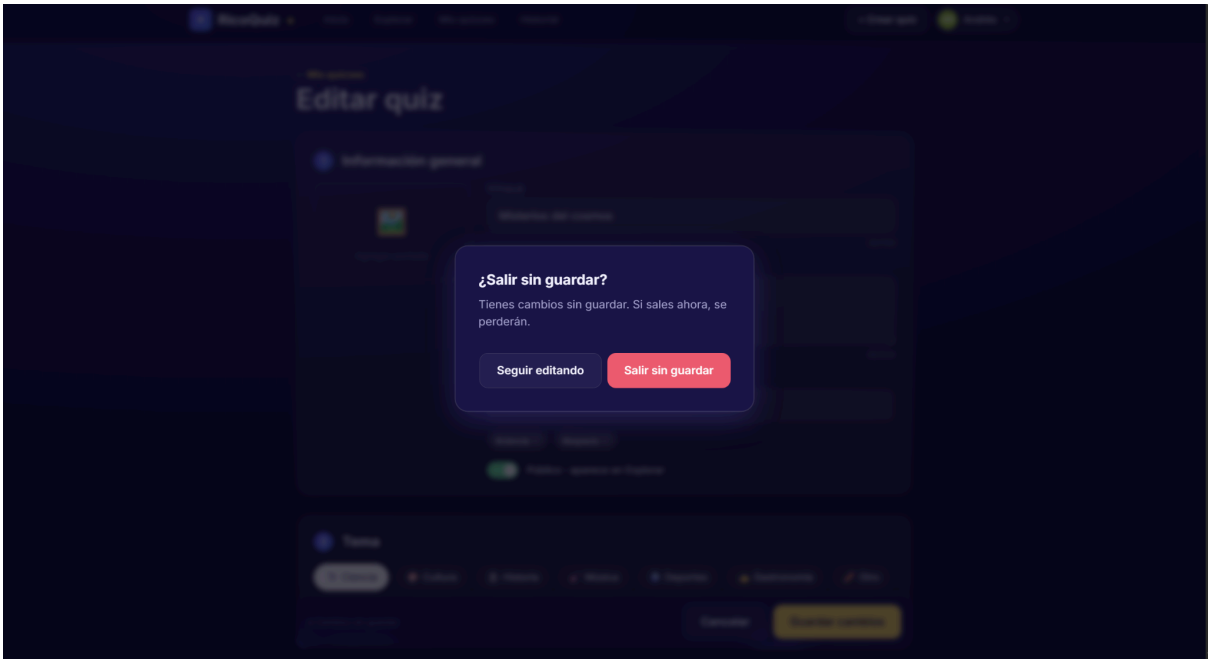
El formulario revisa que esté completo antes de enviar:



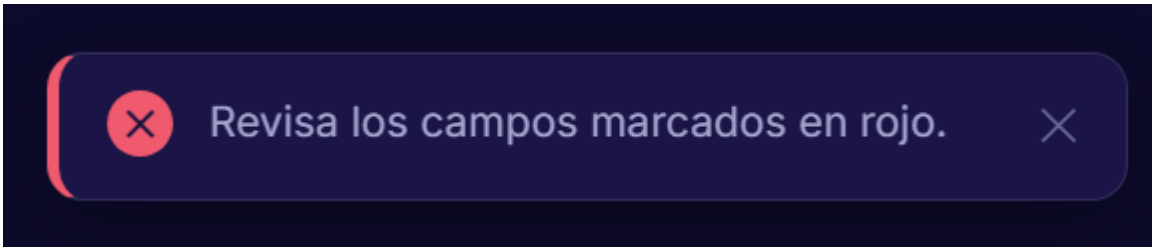
Esto fue agregado después de las capturas, pero también se pueden agregar imágenes a las preguntas, para utilizar una función que está en el backend

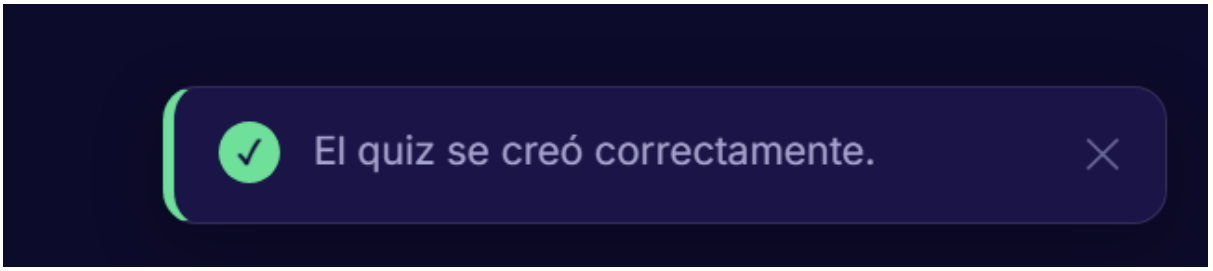


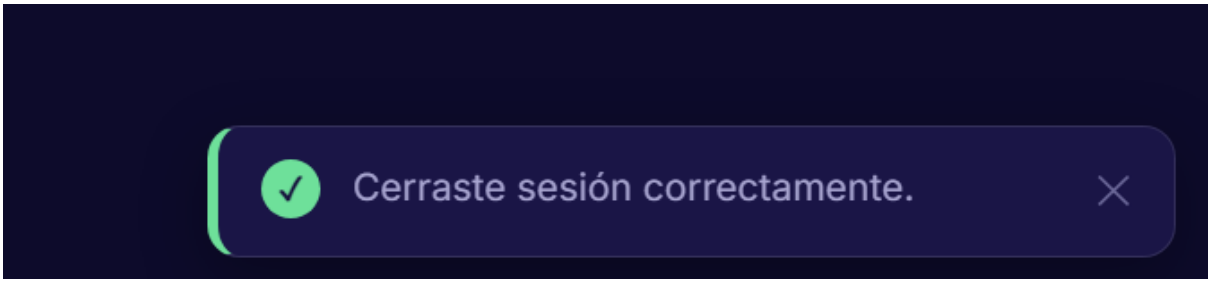
Confirmación de salir de la pantalla sin guardar, tanto que el usuario se intente salir por la navbar o el botón, como por la ruta del navegador



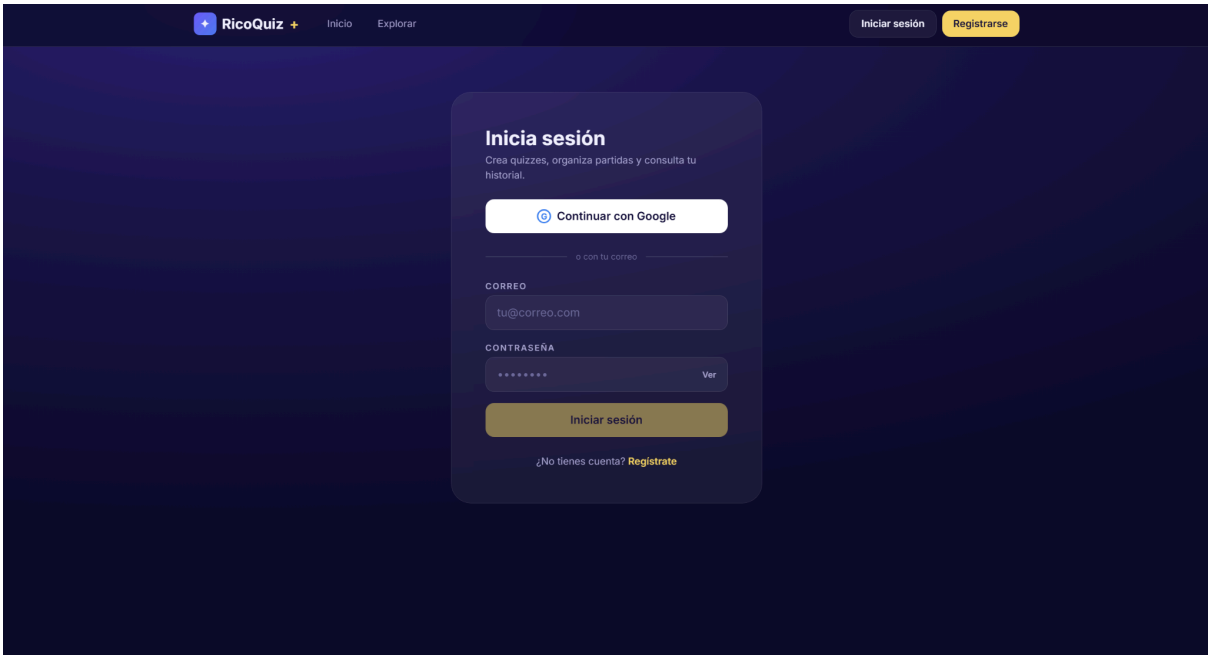
Implementamos una función de notas tipo toast para alertas e información para el usuario, aparece abajo a la derecha de la pantalla



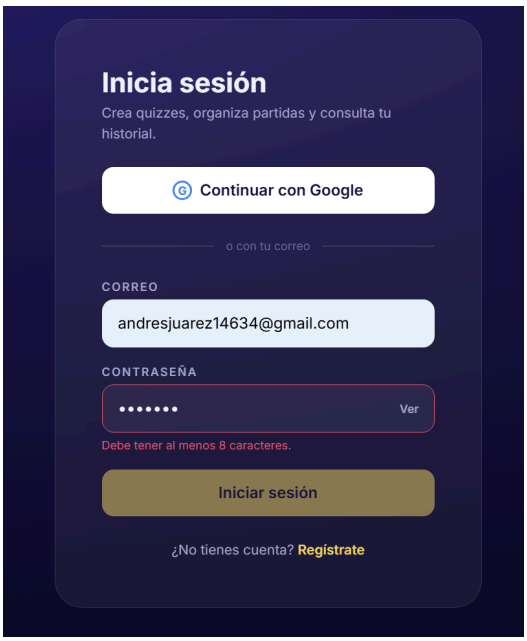


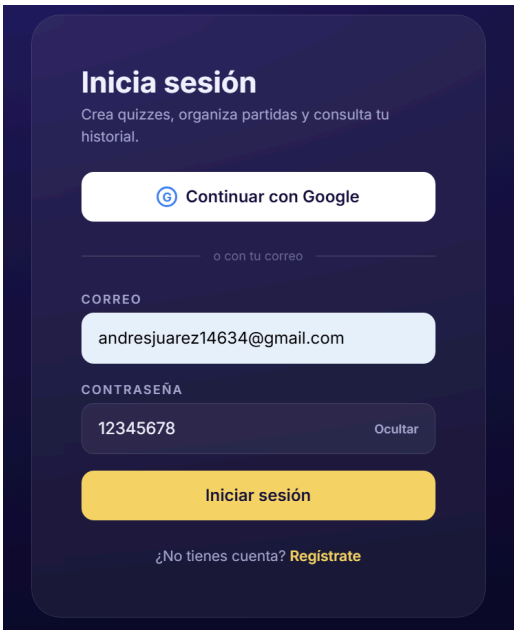


Pantalla de inicio de sesión, y está la opción de google que es el incremento para este proyecto, además se revisa que la contraseña sea de al menos 8 dígitos

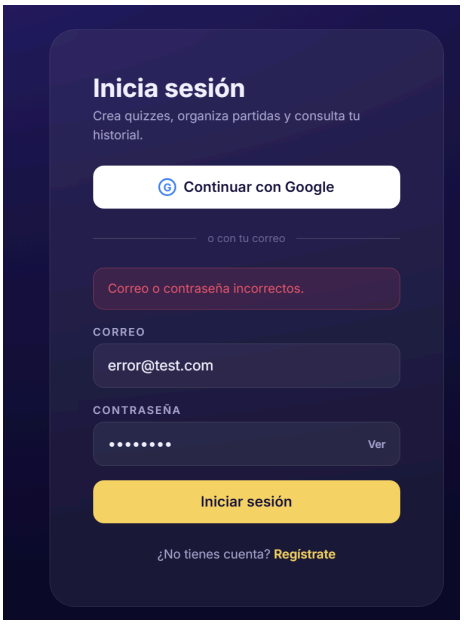


Se puede ocultar y ver la contraseña, y manejo de controles:

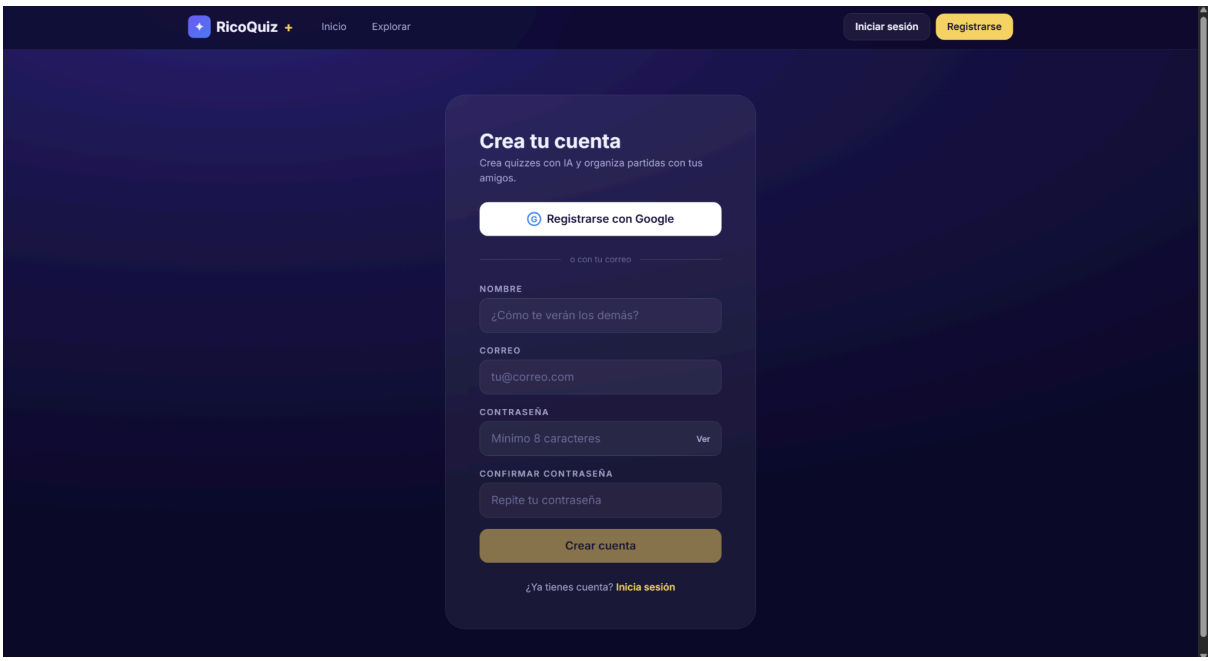




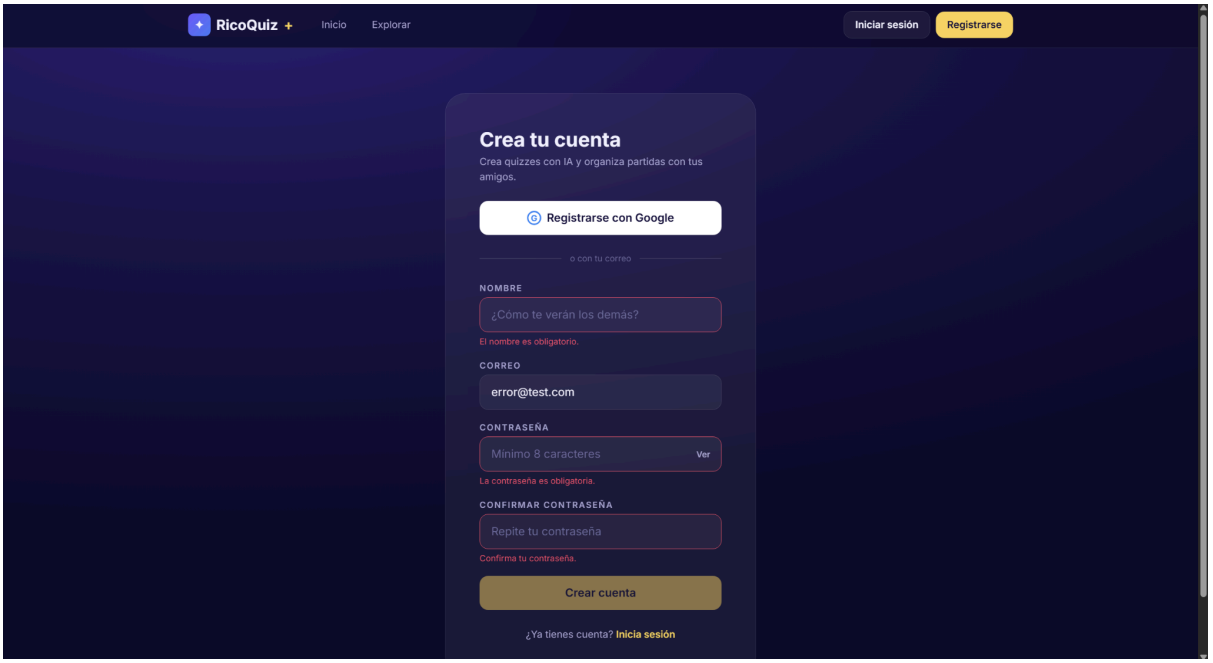
Si se ingresa error@test.com como correo de prueba, forzamos casos de error



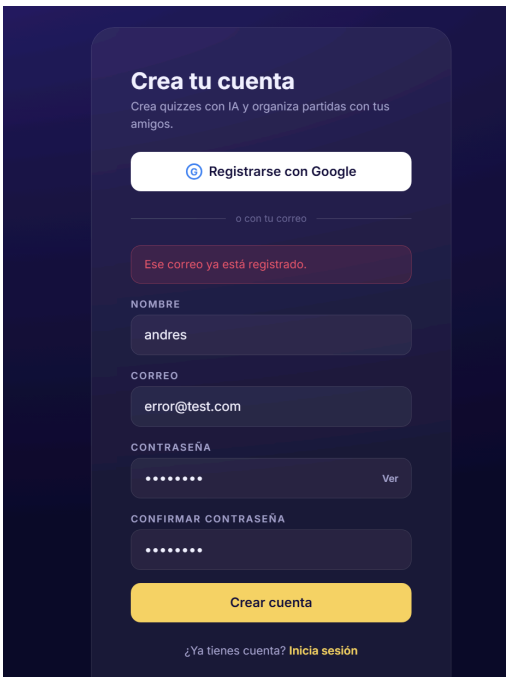
Pantalla de registro:



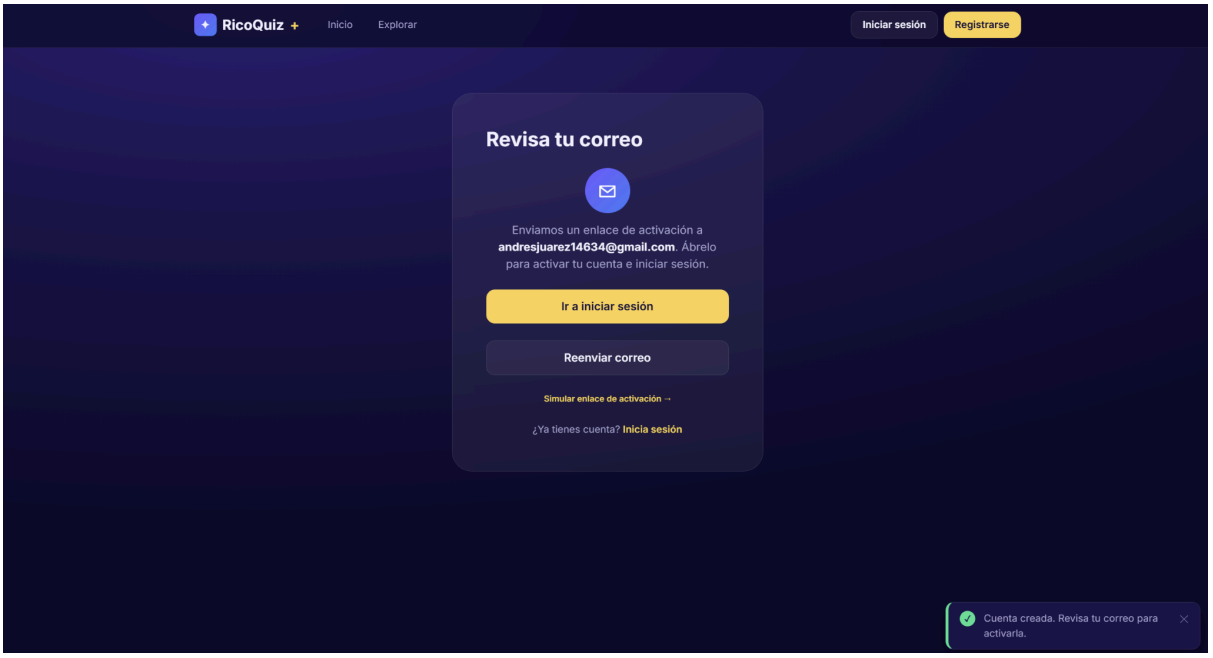
Manejo de casos touched y dirty:



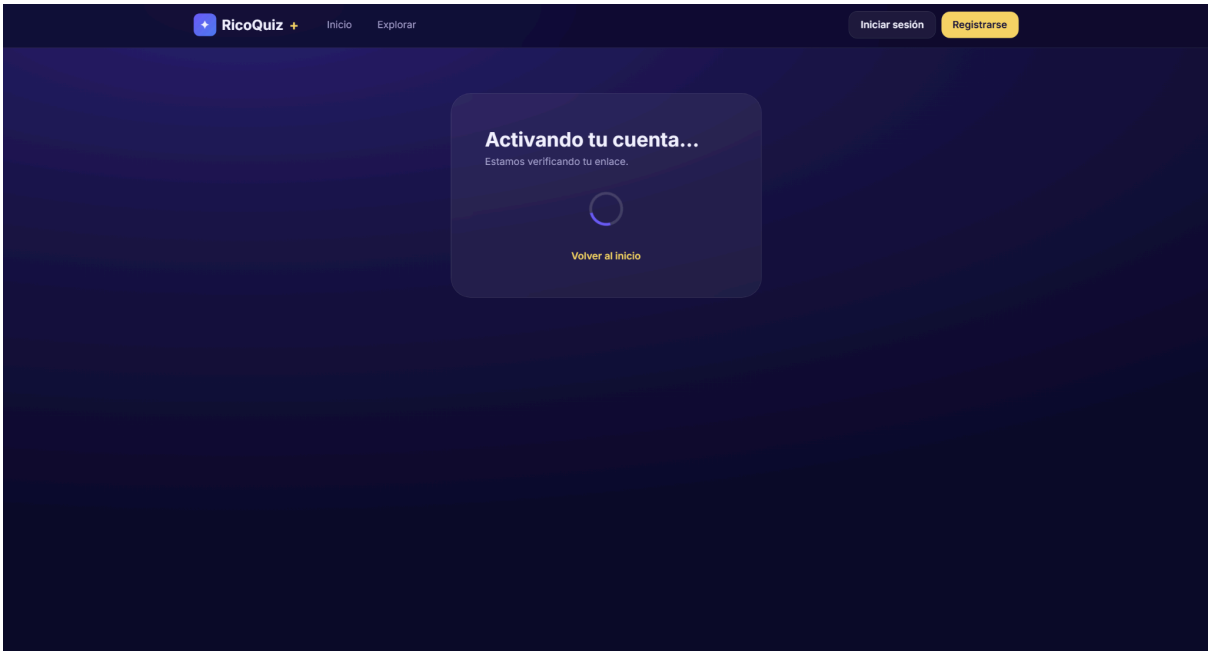
Simulación de caso de error con error@test.com



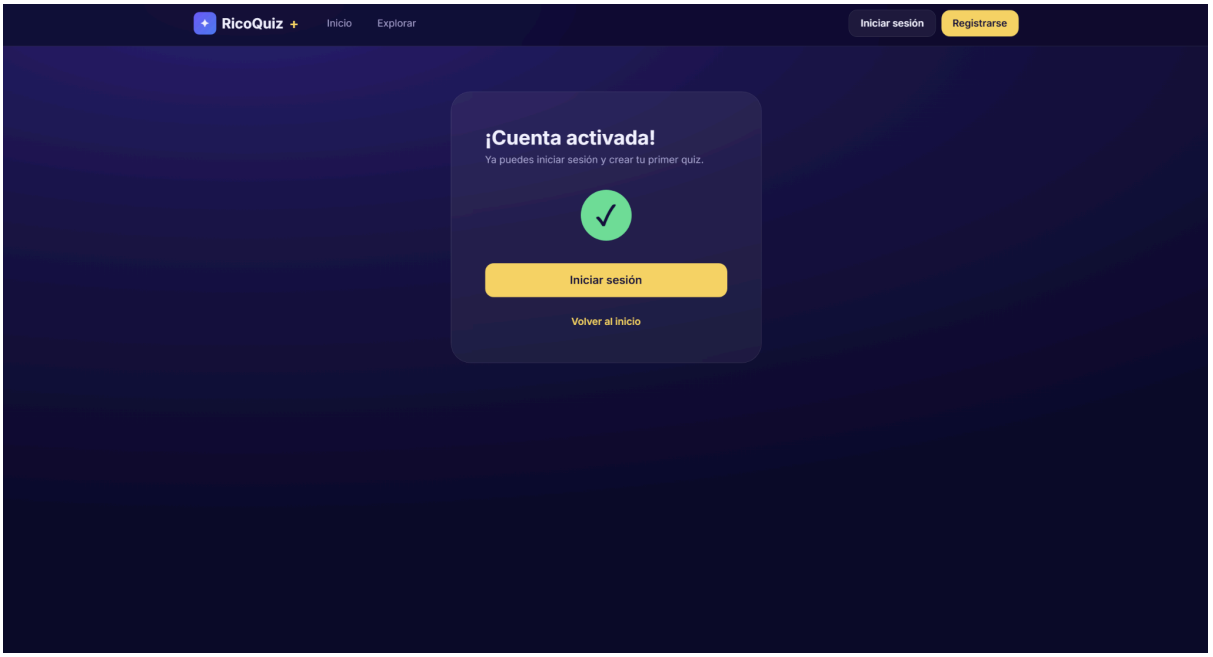
Pantalla de activación de correo una vez creada una cuenta:



Pantalla de cuenta activada con spinner:

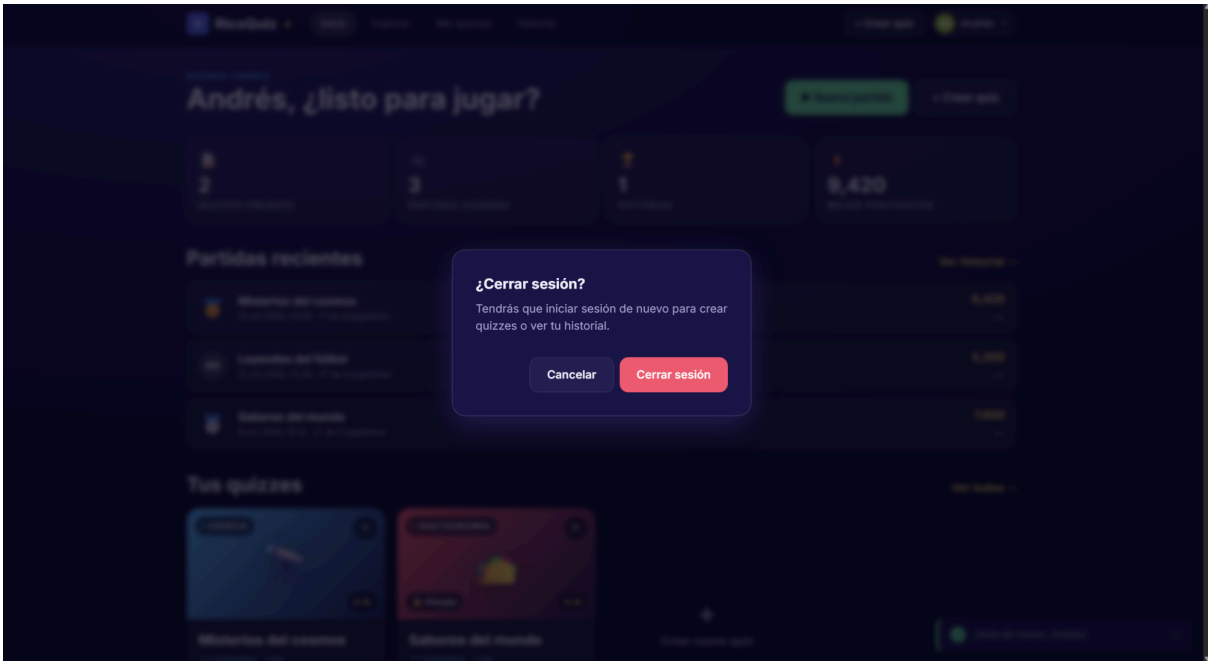


Página con cuenta activada.



Página 404 que hicimos con la sugerencia de Claude porque queríamos hacer algo más vistoso.

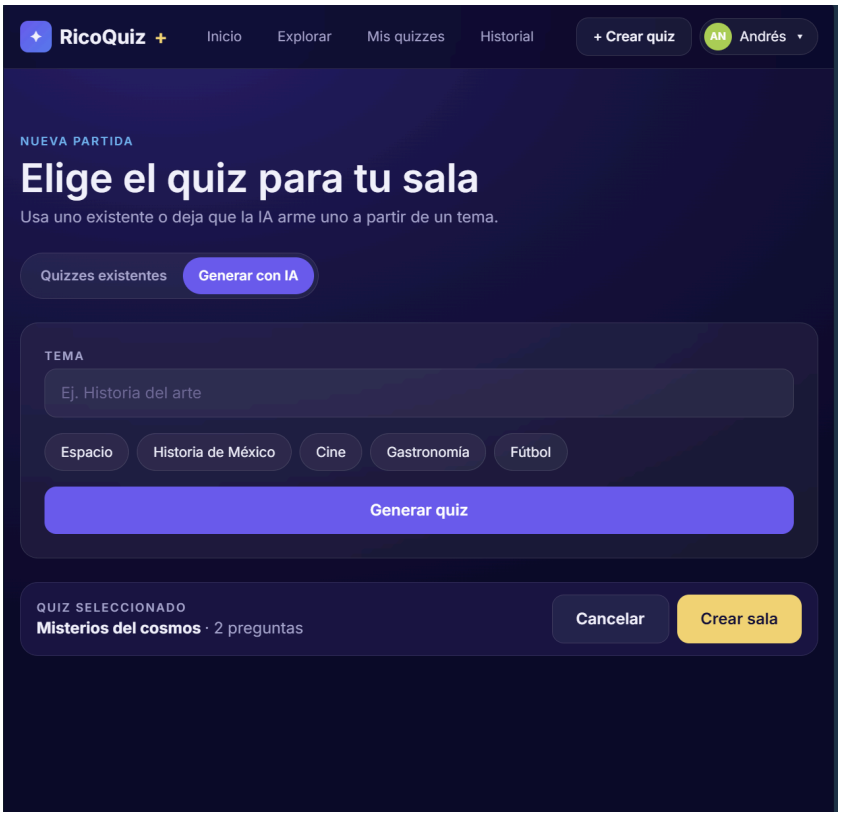
Hicimos un componente para hacer modales confirmación



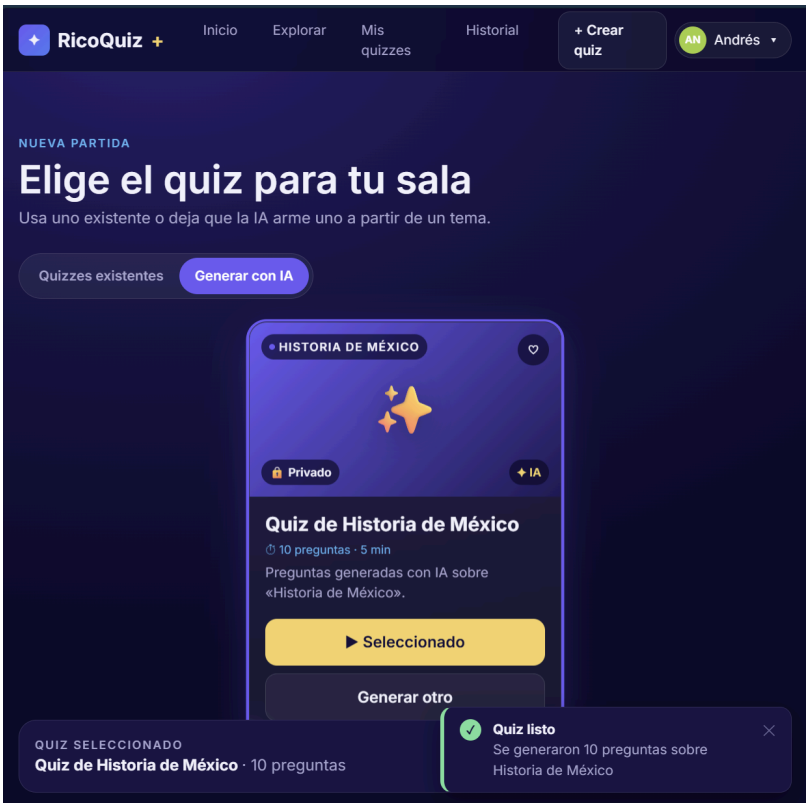
Pantallas de Partida

Elección de partidas ya creadas

Creacion de quizes mediante IA

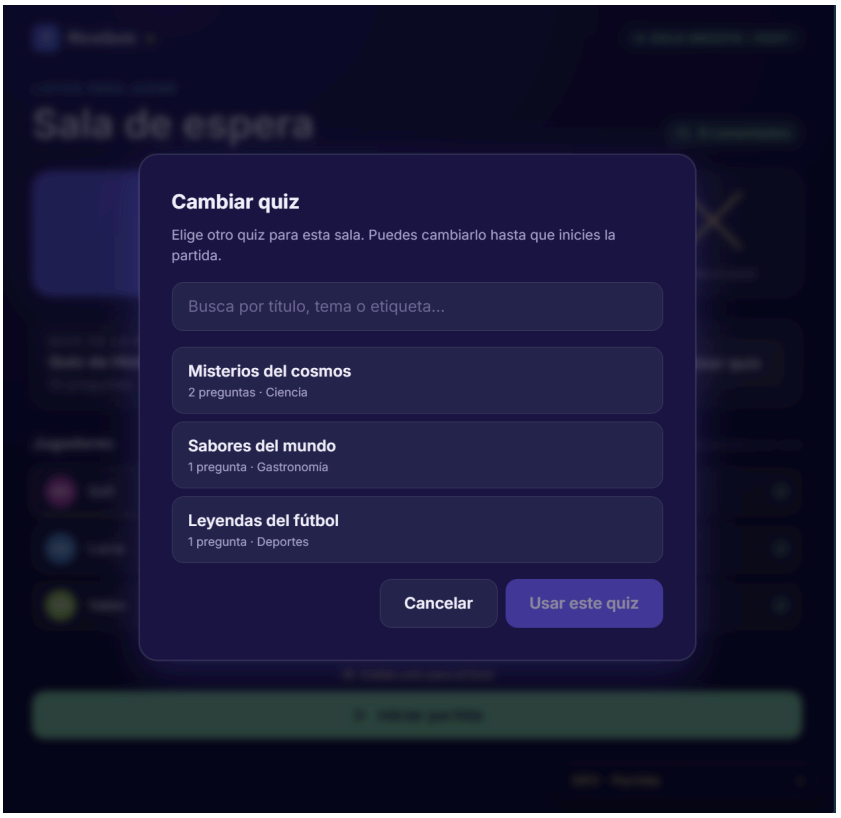


Quiz creado con IA y listo para jugar, con modal de confirmación



Dentro del lobby de juego vista de host, opciones de Dev y modal de confirmacion

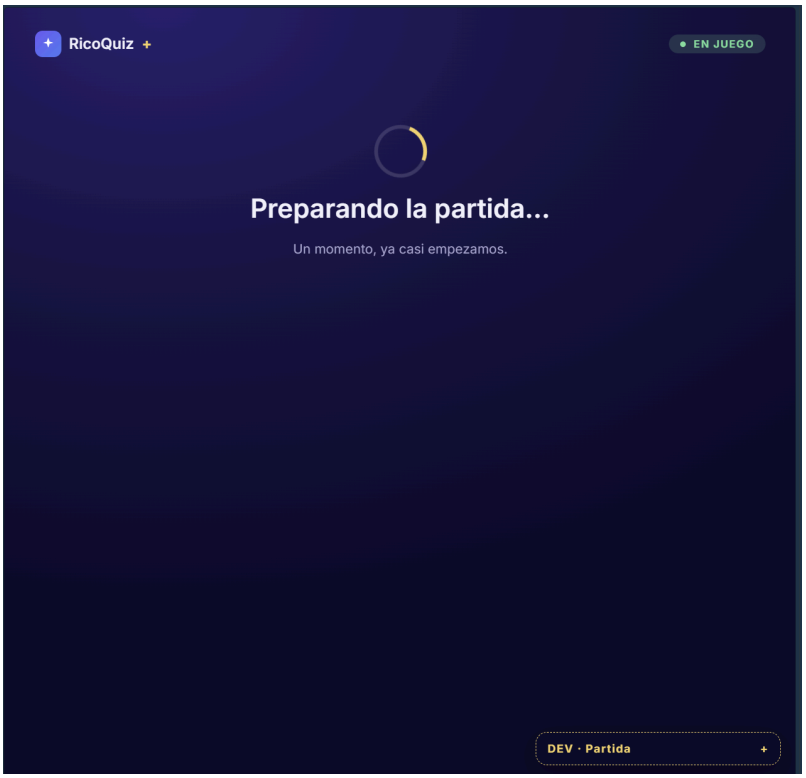
Botón para cambiar quiz en el lobby



Quiz cambiado y modal de confirmación

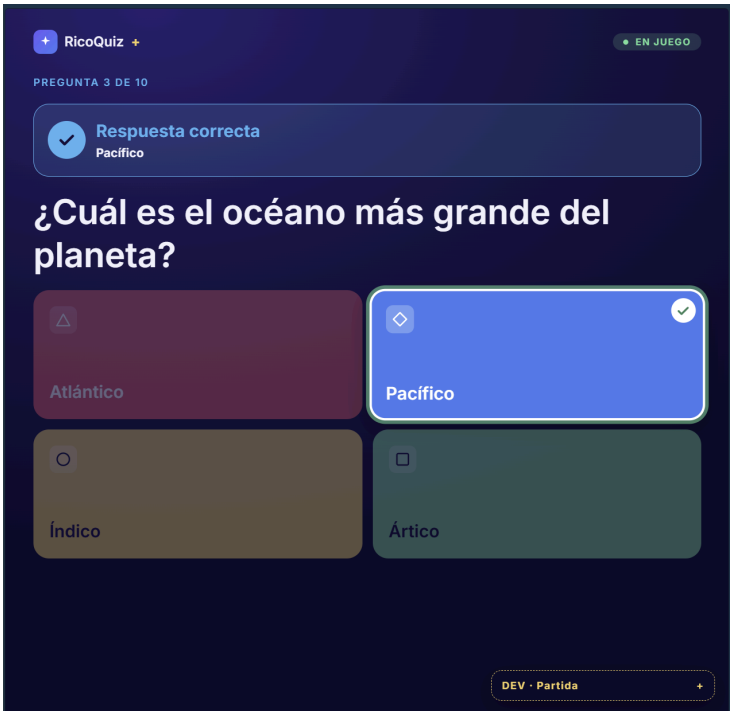
Lobby con vista de jugador

Pantalla de carga antes de mostrar las preguntas para ambas vistas

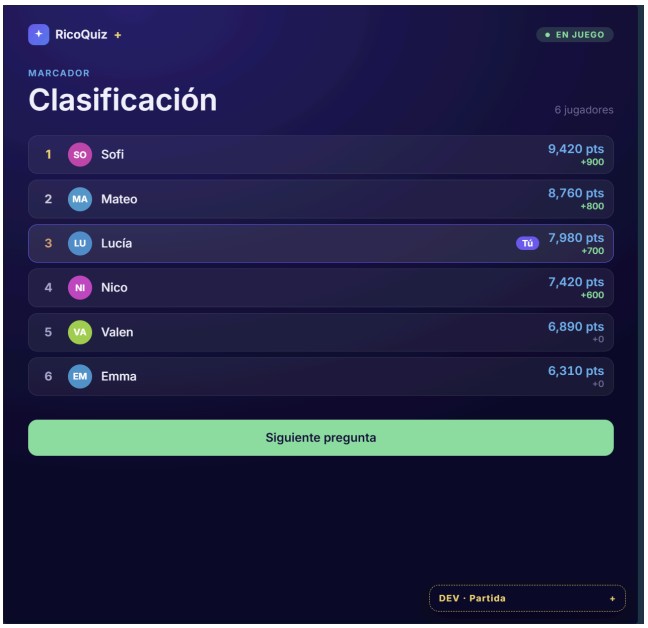


Tiempo para contestar en vista de Host

Así sale la respuesta correcta en ambas vistas

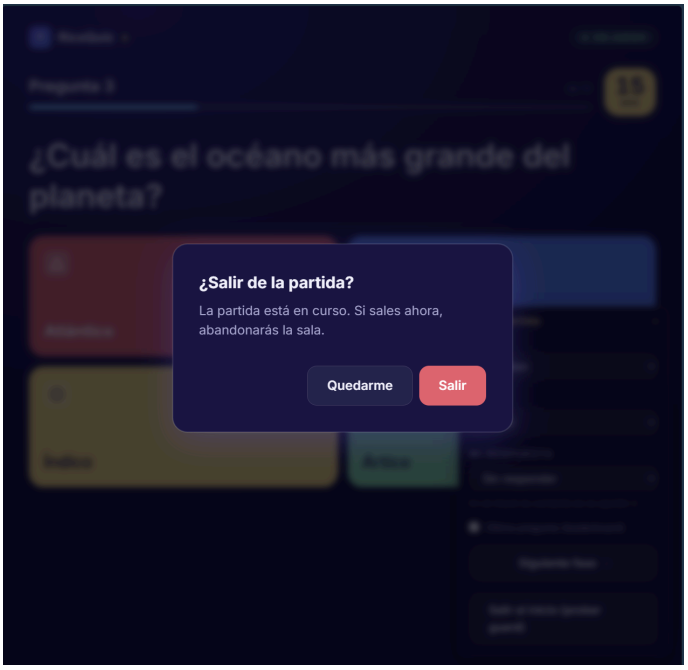


Clasificación con vista de Host, puedes avanzar a la siguiente pregunta



Final de partida en la vista de Host

Si un jugador se intenta salir durante el quiz le saldrá esta alerta, puede continuar o salirse



Pregunta, desde la vista de un jugador

Clasificación con vista de jugador

Final de partida para un jugador

Servicios y guardas Implementadas

Servicios

- AuthService: maneja el estado de sesión (usuario actual y si está autenticado) y las operaciones de login, registro, activación de cuenta, reenvío de activación y logout.
- TokenStorageService: guarda, lee y elimina el JWT en localStorage.
- QuizService: obtiene los quizzes públicos, los del usuario y uno por id. También genera preguntas con IA y guarda quizzes.
- HistoryService: obtiene el historial de partidas del usuario.
- MatchService: verifica si existe una sala con un código dado.
- ToastService: muestra notificaciones temporales de éxito, error, advertencia e información.
- ConfirmService: abre diálogos de confirmación y devuelve si el usuario aceptó o canceló.

Guards

- authGuard: bloquea las rutas privadas a quien no tiene sesión y lo redirige al login. Al iniciar sesión, lo regresa a la página que buscaba.
- guestGuard: impide que un usuario con sesión entre a login o registro, y lo redirige al dashboard.
- unsavedChangesGuard: pide confirmación al salir del formulario de quiz con cambios sin guardar.
- leaveGameGuard: pide confirmación al salir de una partida en curso.

Entrega 3 - APIs y Formularios

En esta entrega el cliente Angular de RicoQuiz+ deja los datos simulados y consume la API propia del proyecto, desplegada en Render, con el Quiz como entidad de CRUD completo, formularios reactivos validados, manejo global de errores, estados de carga, error y vacío, inicio de sesión con Google y despliegue automático en Netlify.

Generales

Campo	Valor
Materia	Tecnologías de Desarrollo en el Cliente
Proyecto	RicoQuiz+ Client - SPA con Angular
Integrantes	Andrés y Adán Juárez
Cliente (Repo)	andresjuar/RicoQuizPlusClient
API (Repo)	andresjuar/RicoQuizPlus
Cliente Desplegado	ricoquiz.netlify.app
API desplegada	proyecto-final-servidor-k6wl.onrender.com
Documentación de la API	Swagger en /api-docs y docs/openapi.json en el repo del cliente
Stack del Cliente	Angular 22, TypeScript, SCSS, Angular CDK, Reactive Forms, Signals, Vitest

Fuente de datos: API propia

Elegimos la API propia de RicoQuiz+ (Node.js, Express, TypeScript, MongoDB/Mongoose, Socket.io), desarrollada en Tecnologías de Desarrollo en el Servidor y desplegada en Render.

- Es el backend real de nuestro proyecto anterior. El juego en tiempo real (Socket.io), la generación de preguntas con IA (Gemini) y las imágenes (Cloudinary) ya viven ahí; una API pública o un BaaS obligaría a duplicar esa lógica.
- Ya tiene contrato documentado con OpenAPI/Swagger, autenticación JWT, validaciones en el modelo y un formato único de error.
- No es simulada, así que no hay que planear un reemplazo posterior.

Para integrar la SPA se hicieron cambios acotados en la API: CORS para el dominio de Netlify, GET /health, GET /quizzes/mine, búsqueda y paginación en los listados (con questionCount en lugar de las preguntas, para no exponer respuestas), login que regresa { token, user }, POST /auth/google, POST /quizzes/generate-questions y la activación por correo como opción configurable.

Contrato de la Api

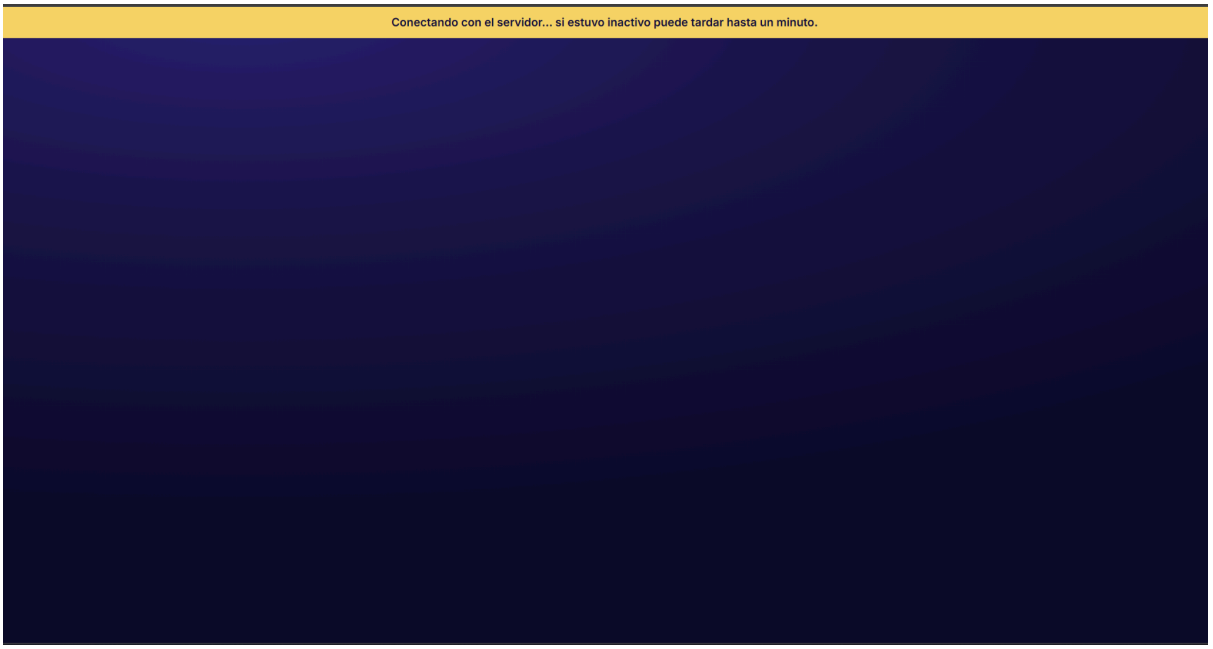
La especificación completa está en docs/openapi.json (OpenAPI 3.0).
También se puede acceder a ella en este enlace:
https://proyecto-final-servidor-k6wl.onrender.com/api-docs

La especificación de los modelos y las respuestas de error se encuentran en este archivo:
https://github.com/andresjuar/RicoQuizPlusClient/blob/main/docs/api-contract.md

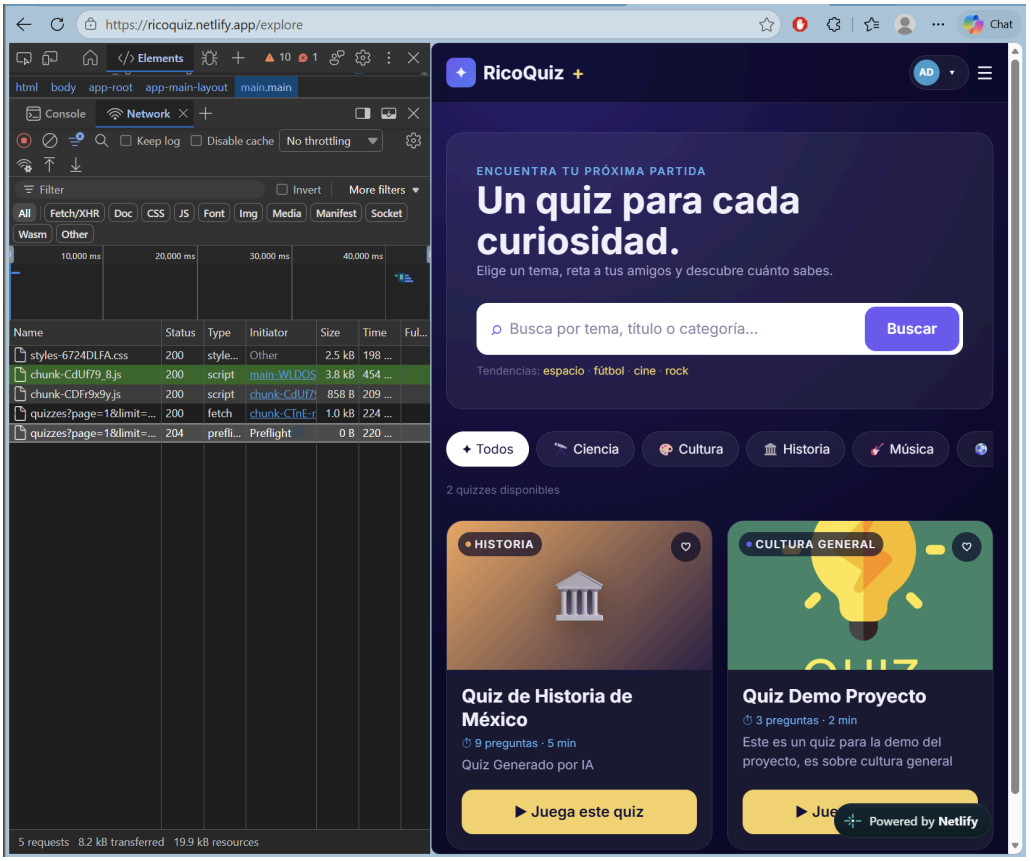
Arquitectura del Cliente

Los componentes nunca hacen peticiones HTTP: piden datos a un servicio, y toda petición pasa por dos interceptores antes de llegar a la API.

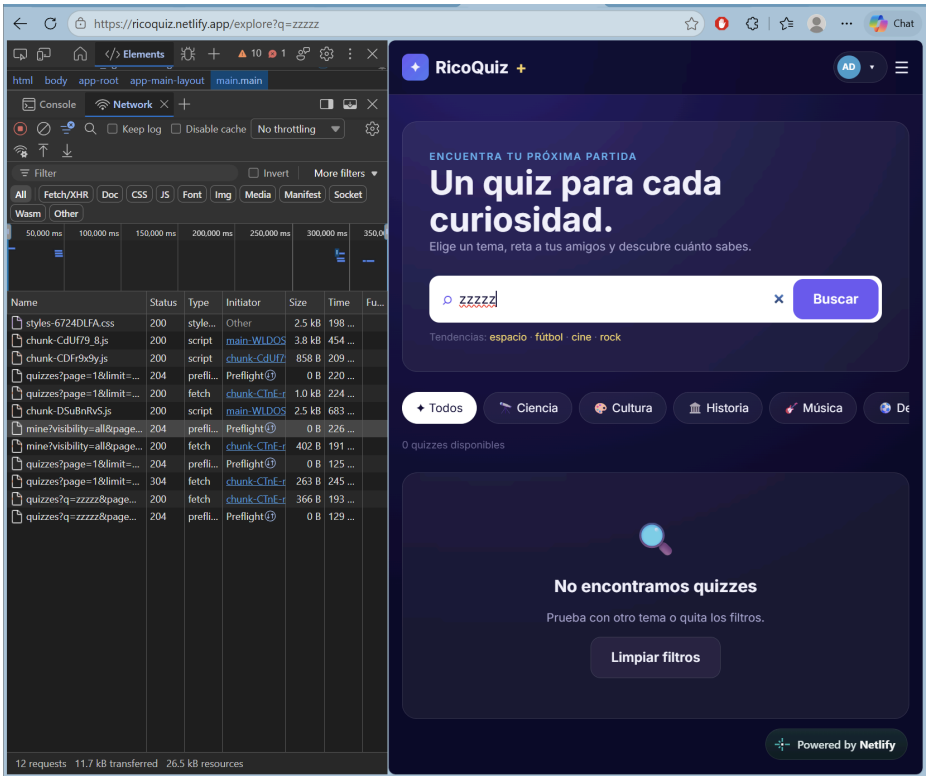
1. Componente (p. ej. Explorar) declara un rxResource con sus parámetros (q, topic, page).
2. Servicio (QuizService, AuthService, HistoryService, MatchService) arma la URL con environment.apiUrl y los query params, y regresa un Observable tipado.
3. authInterceptor agrega Authorization: Bearer <token> solo a peticiones dirigidas a nuestra API.
4. errorInterceptor convierte cualquier error en un ApiError con mensaje para el usuario, muestra el aviso y maneja el 401.
5. <app-view-state> pinta carga, error o vacío según el estado del resource; la página solo pinta los datos.



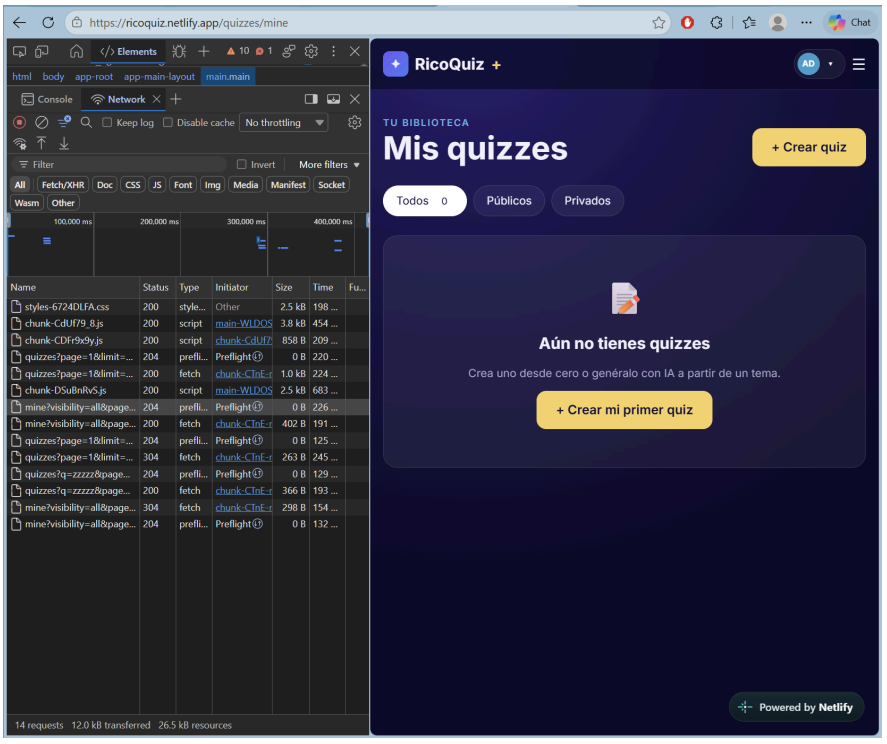
Si se accede a la página y la api en render no está encendida, pues estamos utilizando la versión gratuita, en lo que enciende el api se muestra este mensaje en lugar de que se quede colgado.



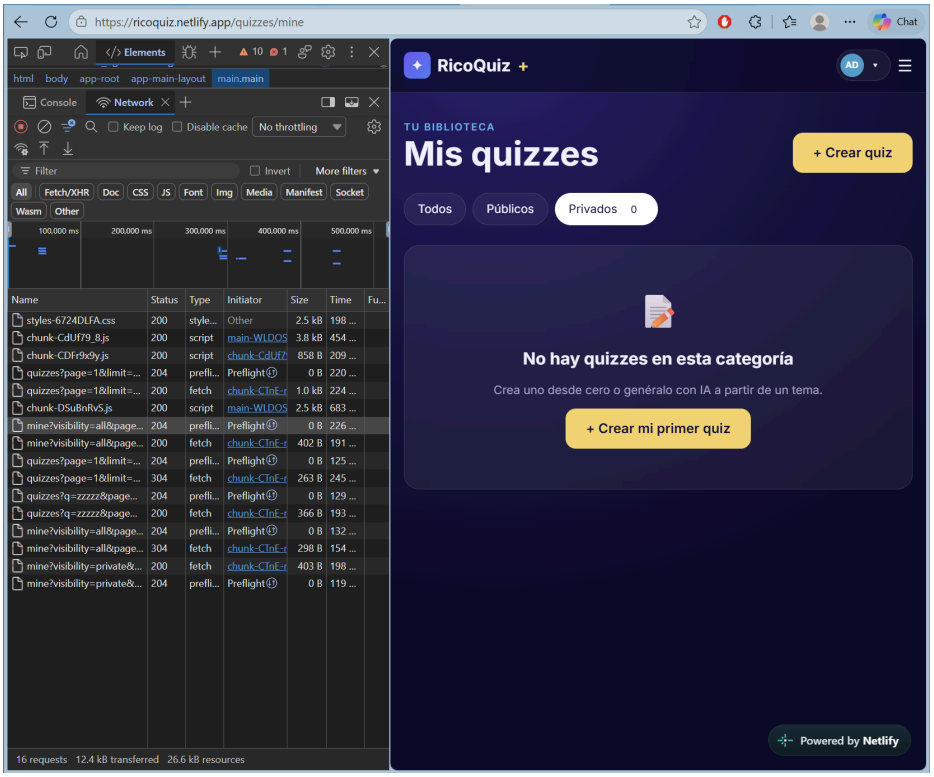
El filtrado lo hace el servidor y solo sale una petición gracias al debounce



No encontramos quizzes con el botón limpiar filtros

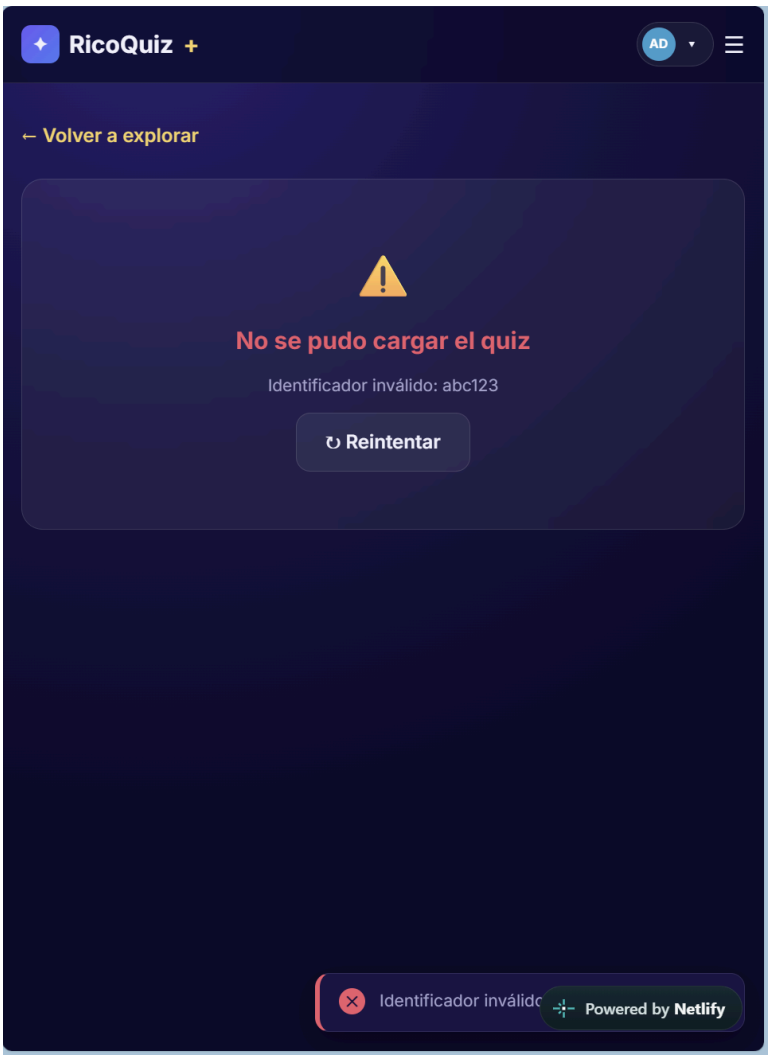


Mis quizzes y sin tener, vemos como al ser todos sale =all

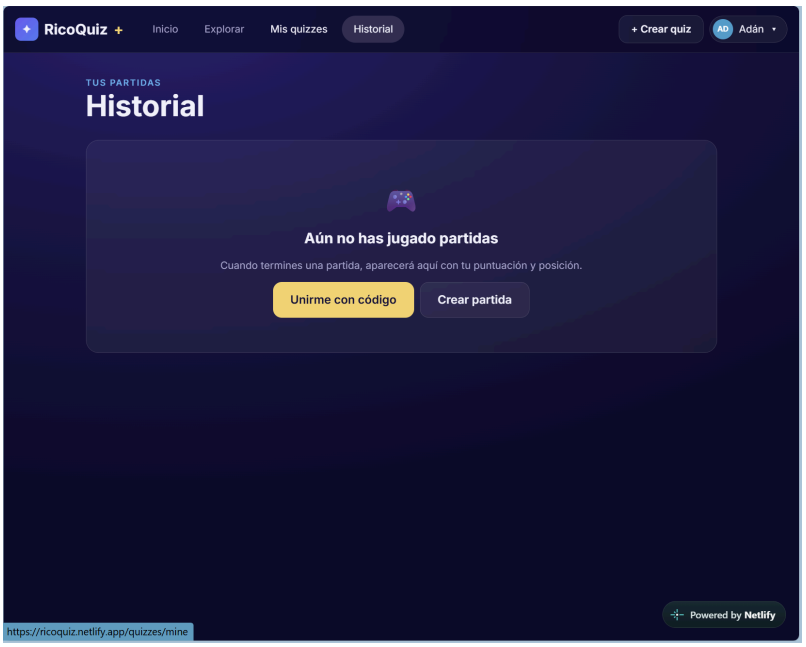


Al ser privados sale =private

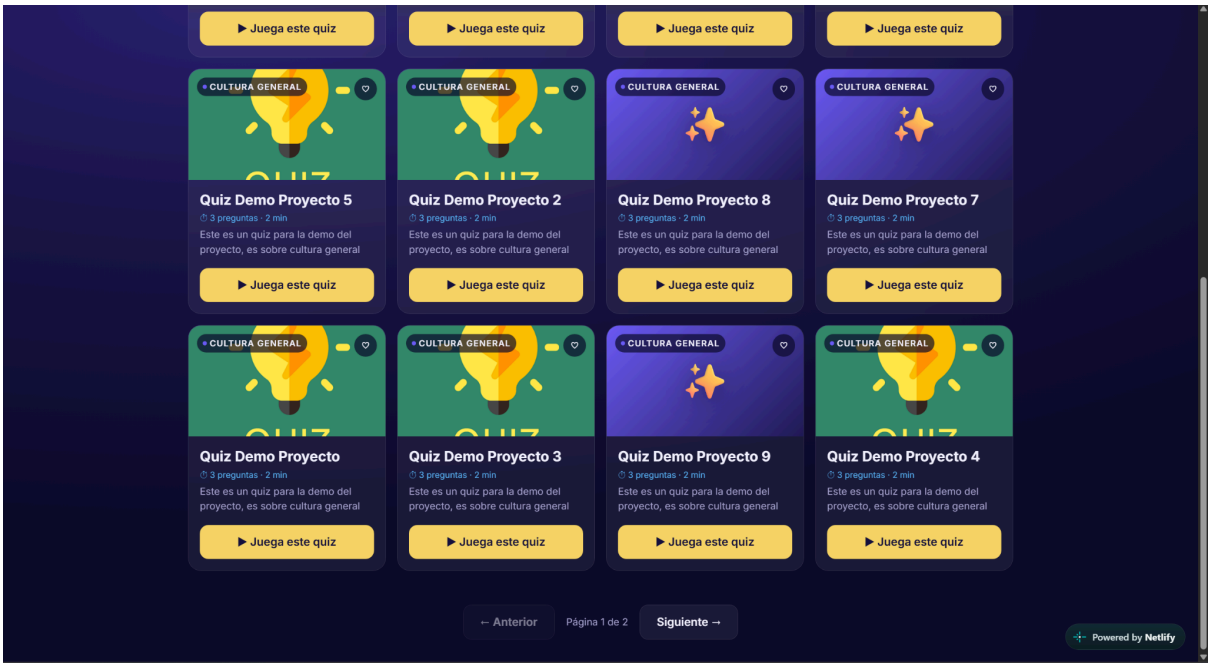
El detalle de editar, eliminar y jugar, si es tu quiz puedes ver las respuestas correctas



Al buscar un quiz con ID invalido manda a esta pantalla y da la opcion de reintentar



En la página de historial, esto nos muestra porque nunca hemos jugado antes



Cuando se llena la página de elementos para la paginación, aparecen botones para cambiar de página

CRUD de Quiz

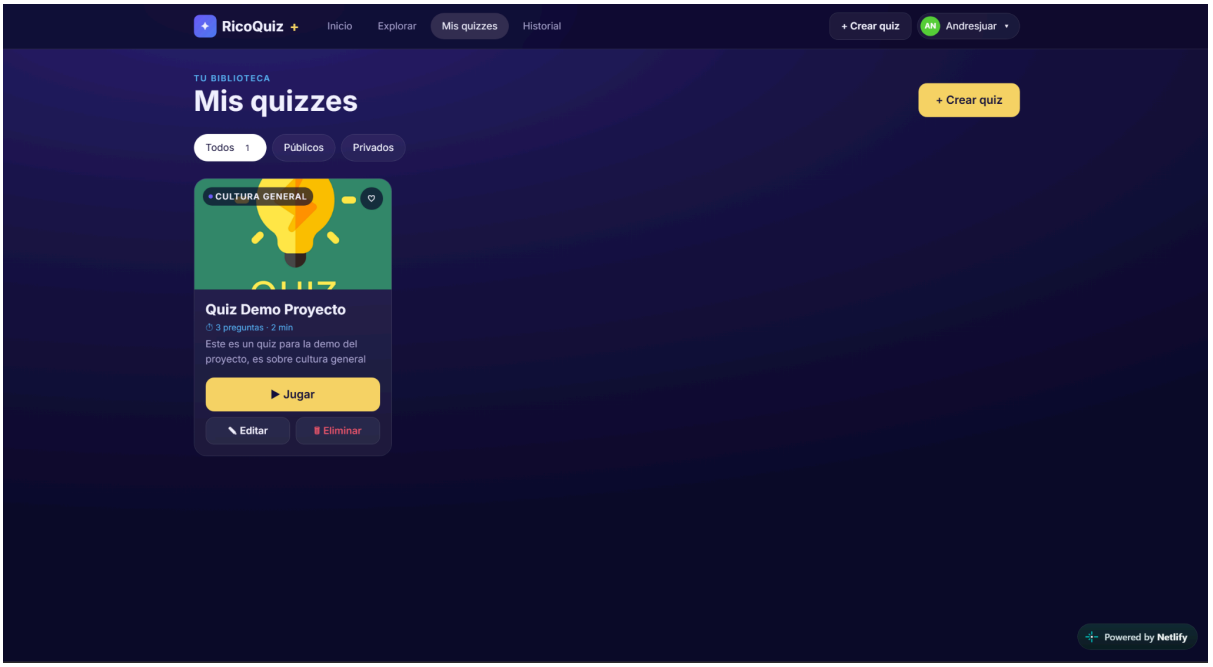
El Quiz es la entidad con CRUD completo.

Operación	Ruta del cliente	Endpoint	Detalle
Listar (públicos)	/explore	GET /quizzes	Búsqueda con debounce de 300 ms, filtro por tema y paginación en el servidor; el estado vive en la URL (?q=&topic=&page=)
Listar (propios)	/quizzes/mine	GET /quizzes/mine	Filtros Todos / Públicos / Privados y paginación
Ver detalle	/quizzes/:id	GET /quizzes/:id	Ruta con parámetro; Editar y Eliminar solo para el dueño

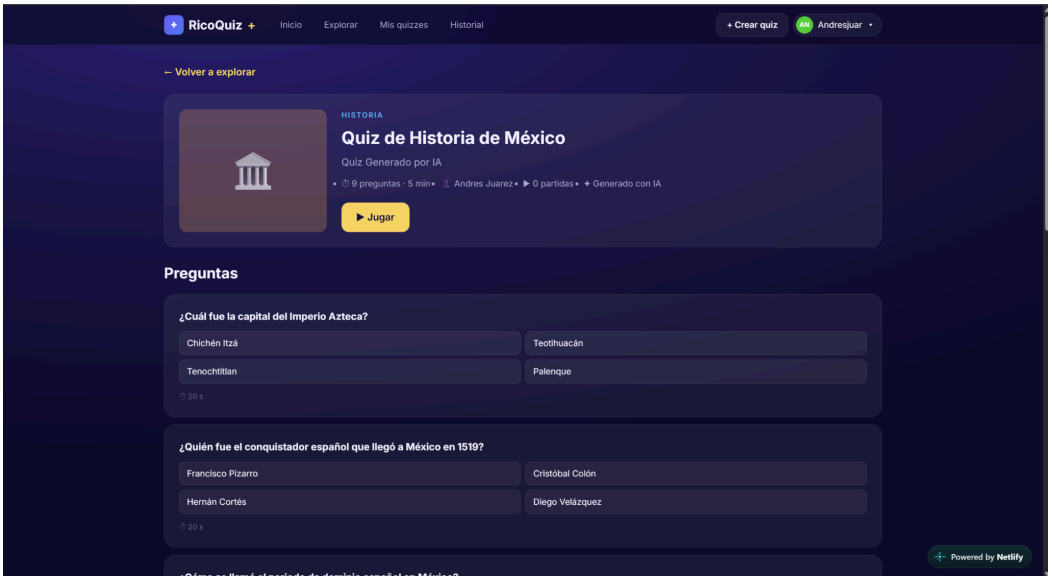
Crear	/quizzes/new	POST /quizzes	Formulario reactivo; preguntas manuales o generadas con IA
Editar	/quizzes/:id/edit	PUT /quizzes/:id	Formulario precargado con el quiz de la API
Eliminar	Mis quizzes y Detalle	DELETE /quizzes/:id	Diálogo de confirmación (Angular CDK) antes de borrar; la lista se recarga con reload()

Listar públicos:

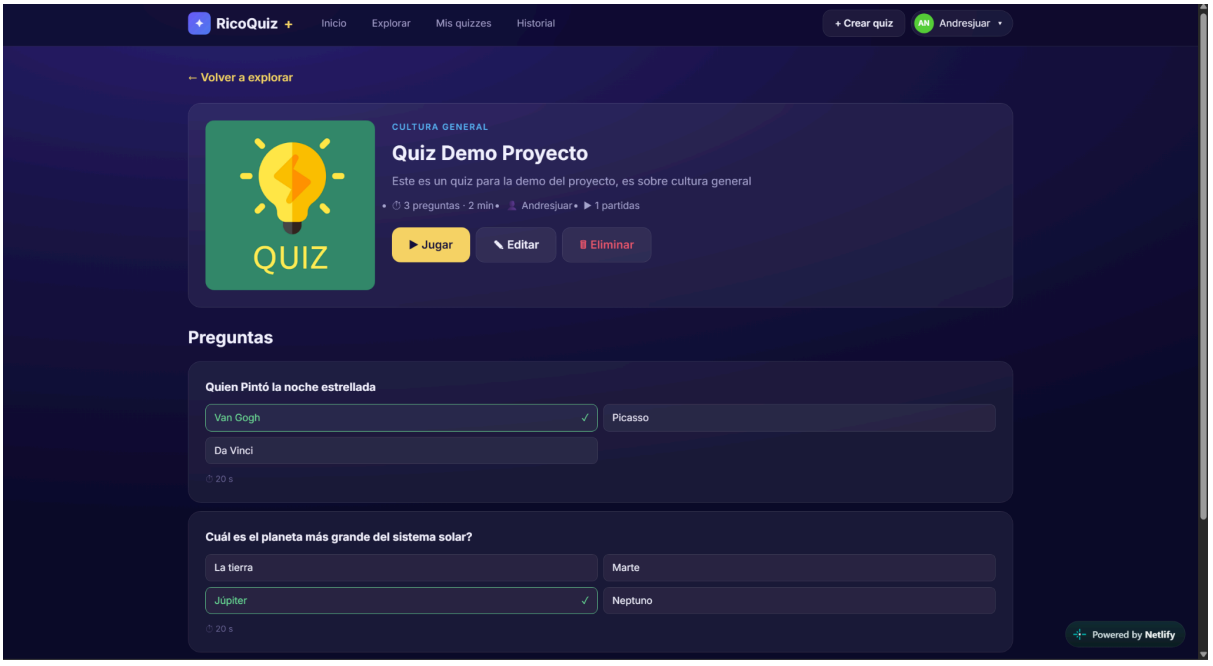
Listar propios


RicoQuiz +
Inicio
Explorar
Mis quizzes
Historial
+ Crear quiz
AN
Andresjuar
TU BIBLIOTECA
Mis quizzes
+ Crear quiz
Todos 1
Públicos
Privados
CULTURA GENERAL
Quiz Demo Proyecto
3 preguntas · 2 min
Este es un quiz para la demo del proyecto, es sobre cultura general
▶ Jugar
Editar
Eliminar
Powered by Netlify

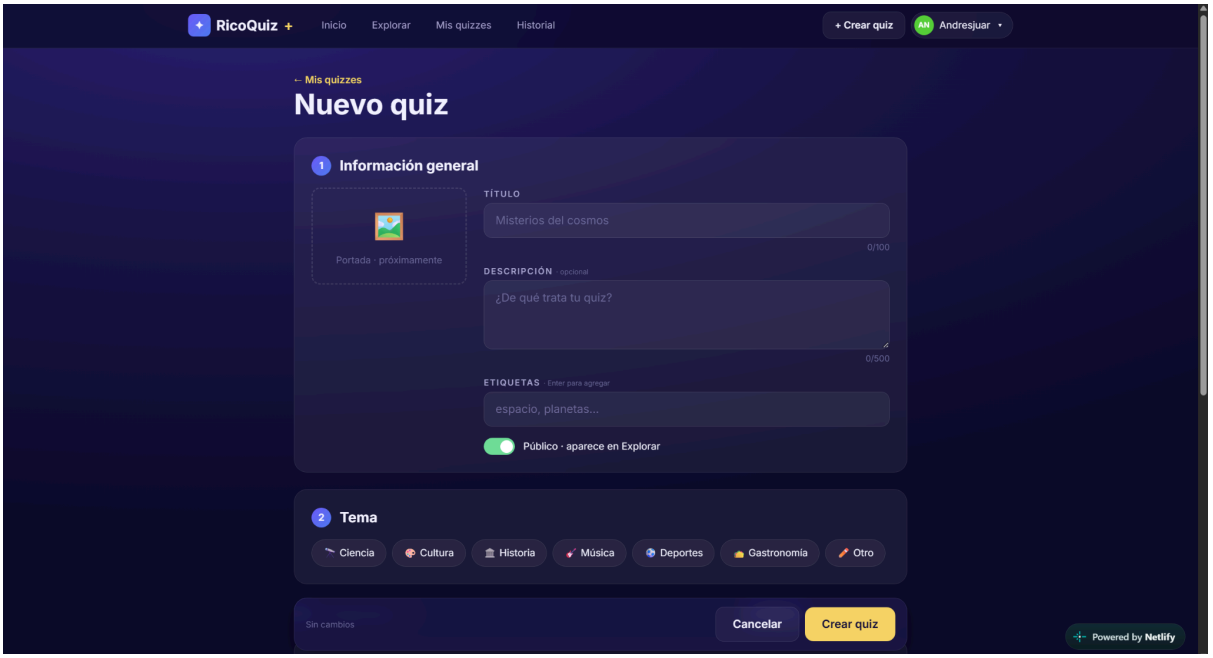
Ver detalle



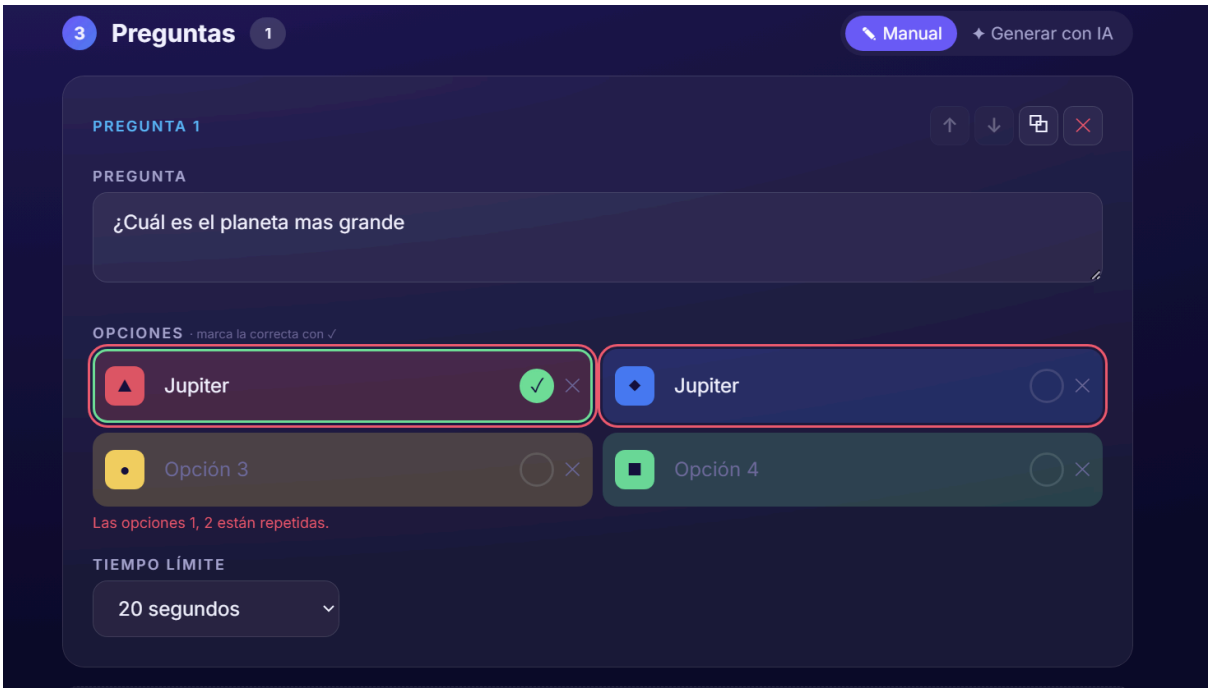
Y si eres dueño te permite editar, eliminar y ver las preguntas con respuesta



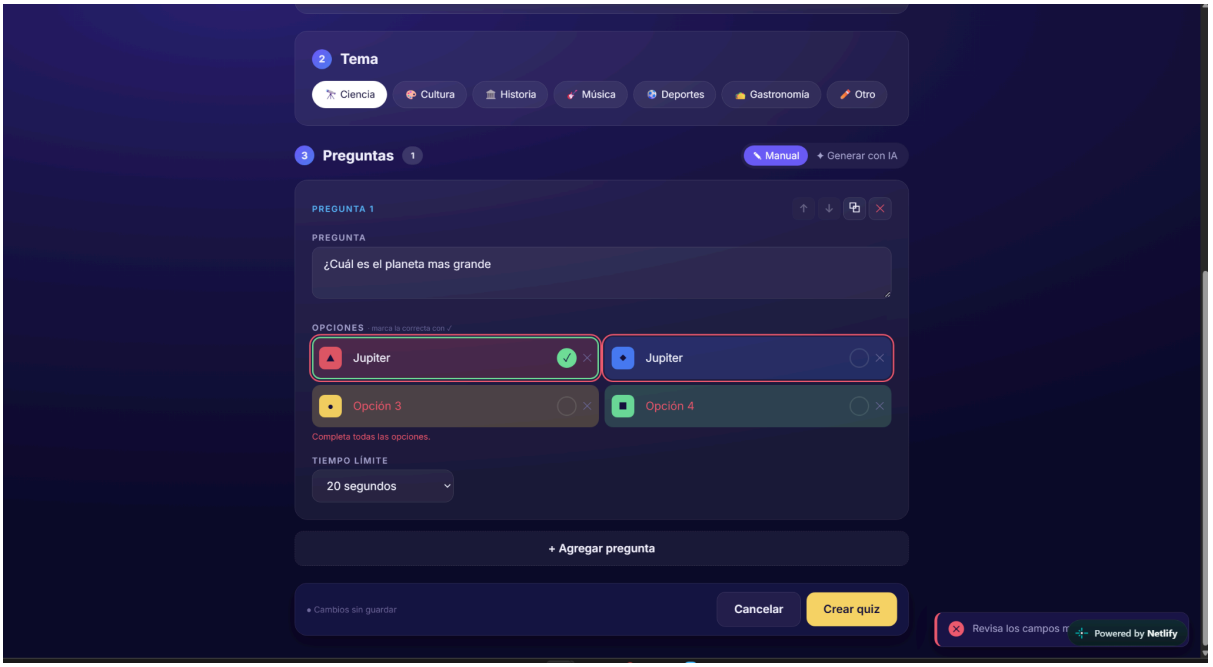
Formulario de Crear Quiz funcionando



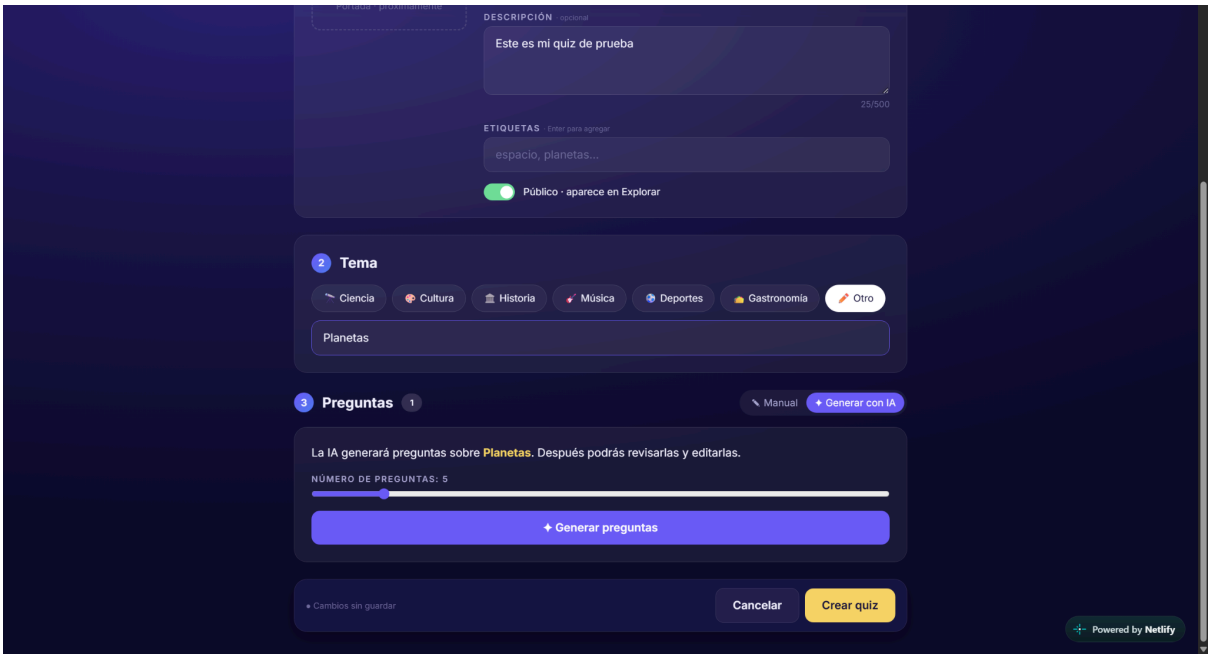
El formulario tiene un validator personalizado:
uniqueOptions: se aplica al FormArray de opciones; marca error si dos opciones son iguales sin importar mayúsculas, acentos ni espacios, y resalta en rojo las repetidas.



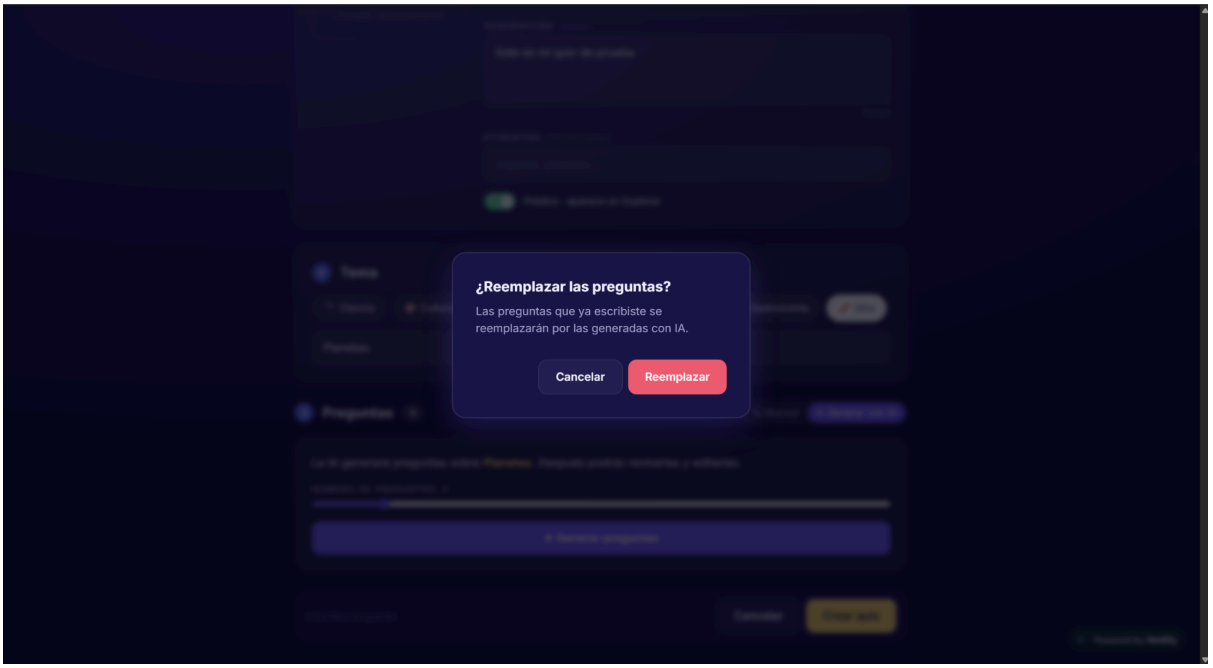
Manejo de errores y muestra con un toast, en este caso como el formulario no está completo no deja crear el quiz



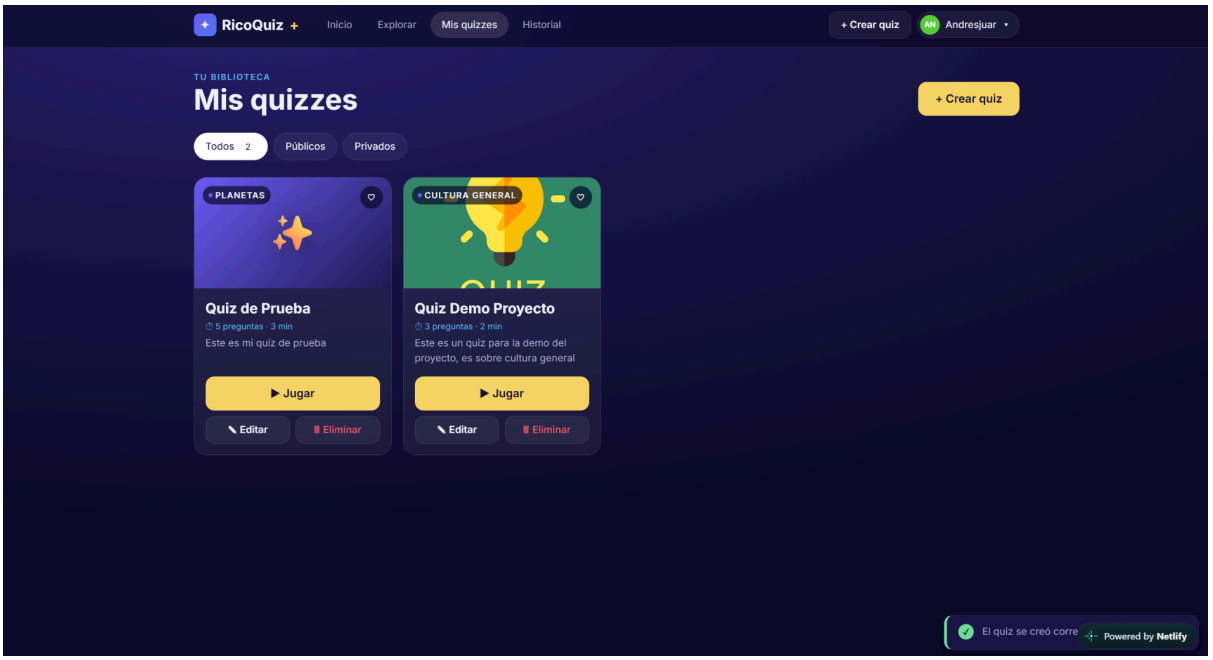
Ya funciona crear preguntas con IA del api



Manejo de opciones con modales

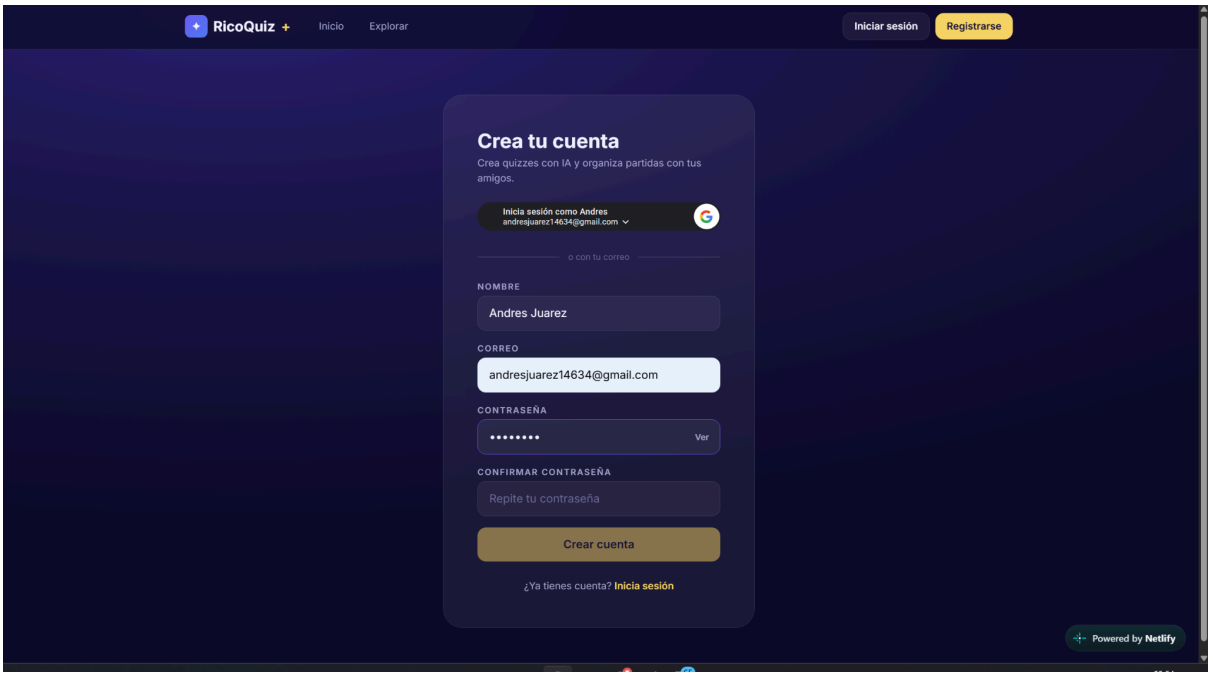


El quiz se crea y queda guardado ya en la base de datos, se muestra una notificación con el toast

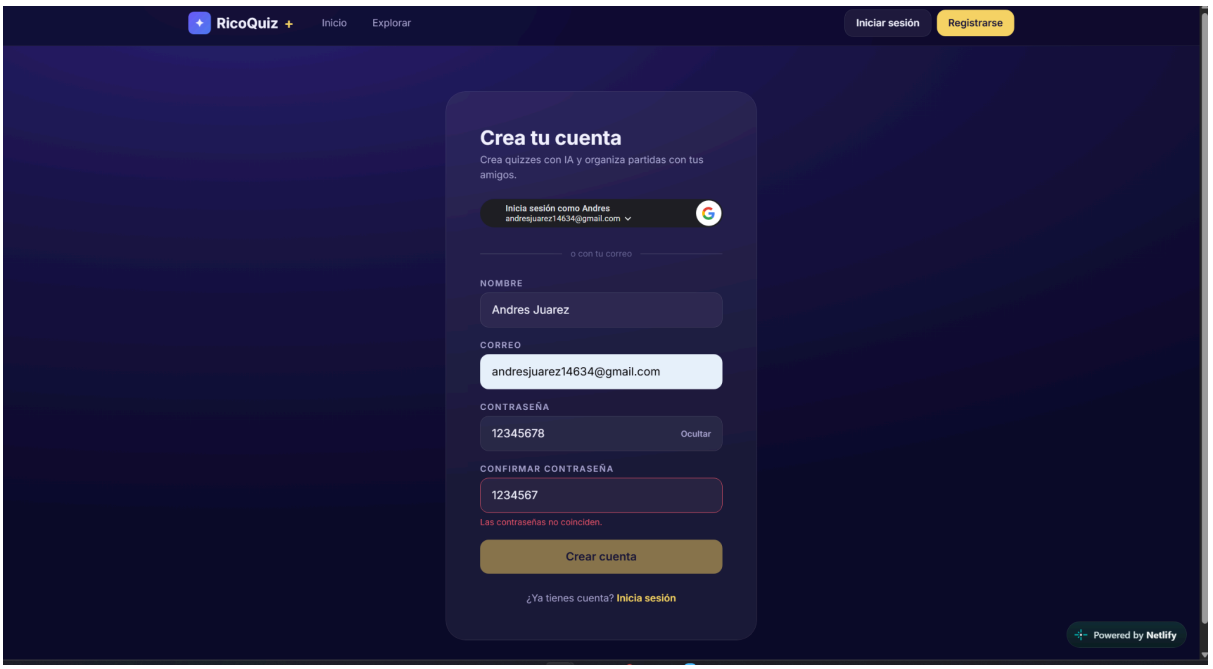


Formularios y validaciones

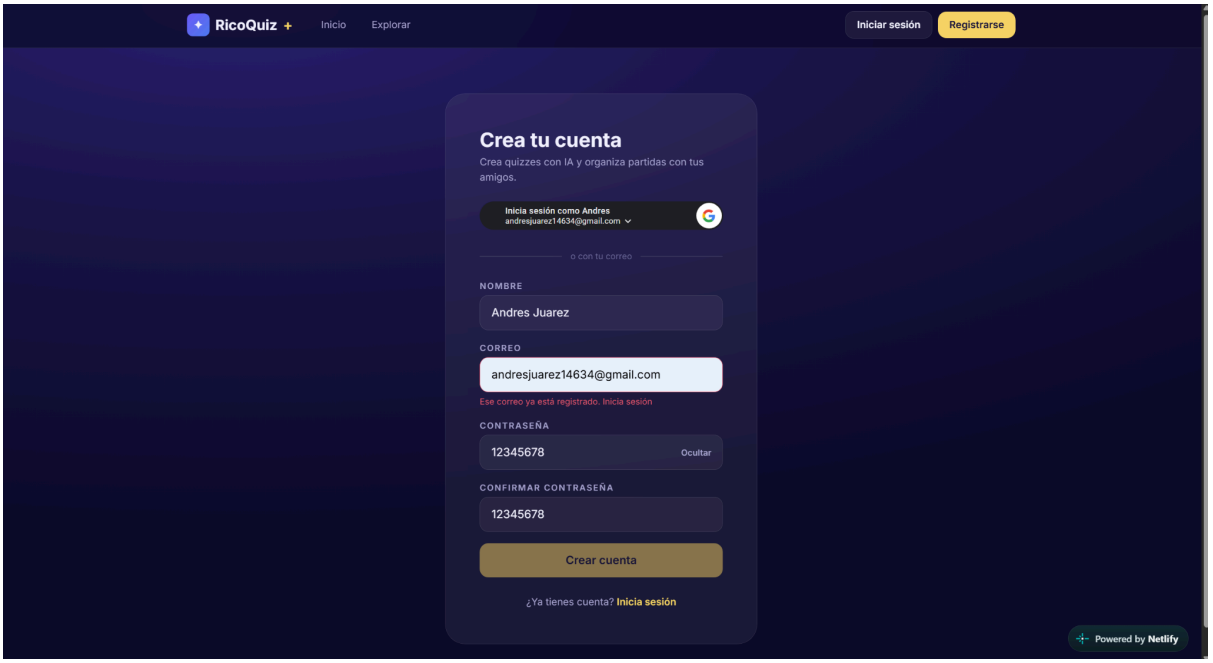
Formulario	Campo	Reglas
Quiz (alta y edición)	Título	Obligatorio, 3 a 100 caracteres
	Descripción	Máximo 500 caracteres
	Tema	Obligatorio
	Preguntas	Al menos 1
	Pregunta	Obligatoria, máximo 300 caracteres
	Opciones	2 a 4, todas obligatorias, **sin repetirse** (`uniqueOptions`)
	Respuesta correcta	Obligatoria
	Tiempo	5 a 120 segundos
	Número de preguntas con IA	3 a 20
Registro	Nombre	Obligatorio, 2 a 50 caracteres
	Correo	Obligatorio, formato válido; si la API dice que ya existe, el error aparece en este campo
	Contraseña	Obligatoria, mínimo 8 caracteres
	Confirmación	Debe coincidir (`passwordsMatch`)
Inicio de sesión	Correo y contraseña	Obligatorios, formato de correo, mínimo 8 caracteres
Unirse a sala	Código	Exactamente 4 letras



Botón de crear cuenta deshabilitado si no se llenan todos los campos

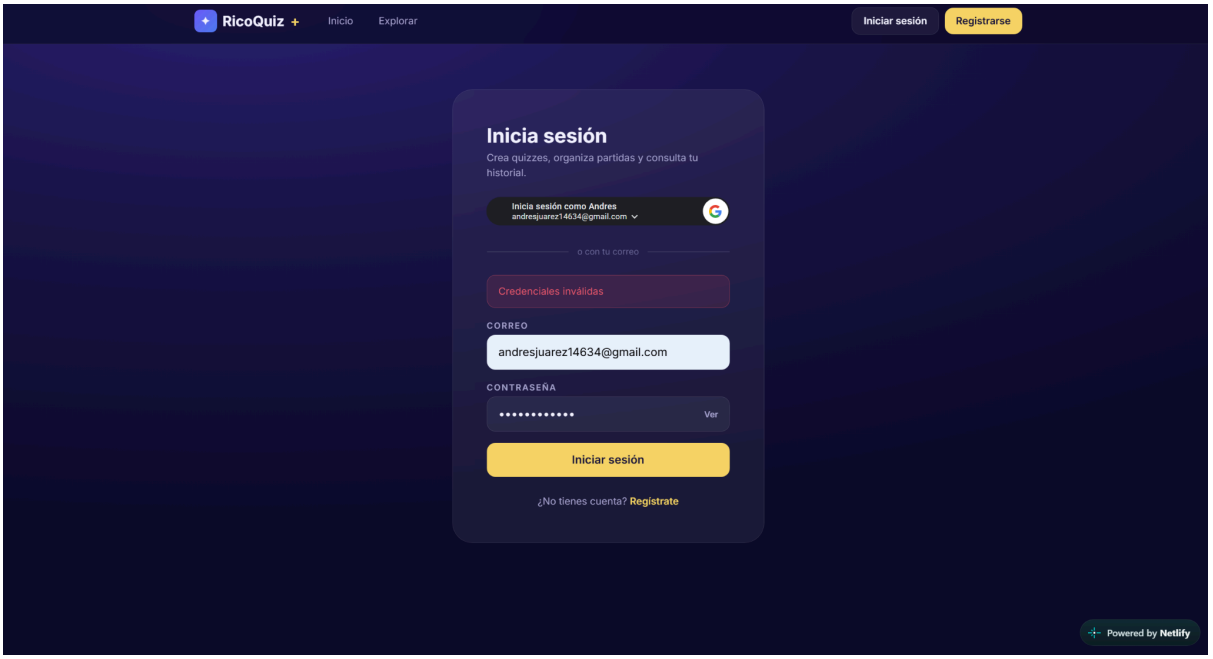


Se muestran mensajes de error por campo, en este caso las contraseñas no coinciden.



Otro error de campo, se intenta crear una cuenta pero el correo ya existe.

Formulario de inicio de sesión con errores:

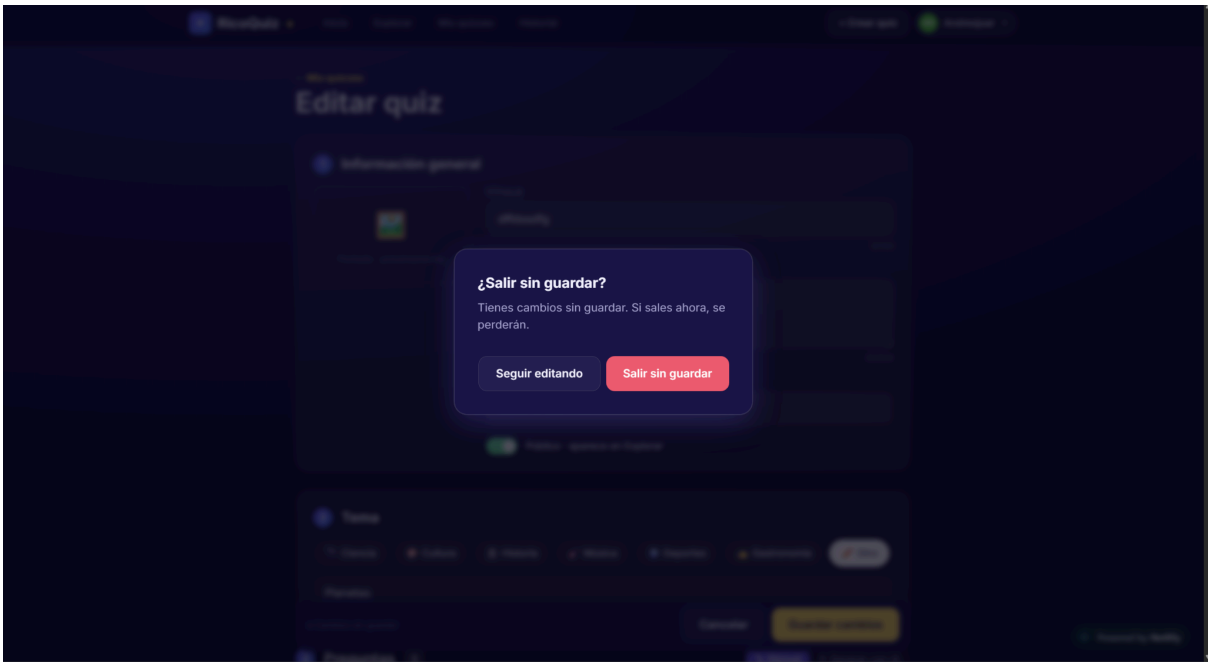


Formulario de código de partida:

Validadores personalizados (src/app/shared/validators):

- uniqueOptions: se aplica al FormArray de opciones; marca error si dos opciones son iguales sin importar mayúsculas, acentos ni espacios, y resalta en rojo las repetidas.
- passwordsMatch: se aplica al FormGroup de registro y compara contraseña y confirmación.

Guard de salida. unsavedChangesGuard (canDeactivate) pide confirmación si el formulario tiene cambios sin guardar, y beforeunload avisa al cerrar o recargar la pestaña.



Estado del envío. El botón muestra "Guardando…" y queda deshabilitado; al terminar aparece un toast de éxito, y si la API falla el botón se vuelve a habilitar y el interceptor muestra el motivo. (Como la imagen de la sección anterior)

Estados de la interfaz y manejo de errores

Toda vista que consume la API usa el componente <app-view-state> junto con rxResource, así los cuatro estados se resuelven igual en todas las páginas.

Estado	Cuándo	Qué ve el usuario

Carga	Aún no hay datos	Skeleton de la página; si tarda más de 5 s, "Despertando el servidor…" (Render gratuito se duerme)
Error	La petición falló	Mensaje de la API y botón Reintentar (resource.reload())
Vacío	La lista llegó sin elementos	Ícono, título y texto propios de cada vista
Éxito	Al crear, editar o eliminar	Toast de confirmación

Manejo global de errores. errorInterceptor es el único lugar que decide qué hacer con un error HTTP:

- Traduce HttpErrorResponse a ApiError con el mensaje en español que manda la API.
- 401 con sesión abierta: cierra sesión, avisa y redirige al login conservando la página de regreso.
- Status 0 o 502–504: avisa que el servidor no responde o está despertando.
- 500: mensaje genérico.
- Las peticiones cuyo error ya pinta el formulario (login, registro, activación) se marcan con un HttpContextToken para no mostrar el mensaje dos veces.

Los componentes solo usan el callback de error para apagar indicadores de carga; ninguno decide mensajes ni redirecciones.

Autenticación

Aunque la rúbrica no exige conectar el flujo de sesión en esta entrega, ya funciona completo.

Inicio de sesión con Google. El cliente carga Google Identity Services y muestra el botón oficial en la landing, el login y el registro. Al elegir una cuenta, Google entrega un ID token (credential) que el cliente manda a POST /auth/google. La API lo verifica con google-auth-library contra su Client ID y:

- inicia sesión si ya existe un usuario con ese googleId;
- vincula la cuenta si ya existe un usuario local con el mismo correo verificado;
- crea una cuenta de Google (sin contraseña, ya activa) si el correo no existe.

El flujo de ID token no necesita client_secret ni sesiones del lado del servidor, y la sesión sigue siendo el mismo JWT de la API.

La activación por correo se quedó desactivada de momento ya que render en la versión gratuita no permite el envío de correos

Sesión: El JWT se guarda en localStorage (TokenStorageService); authGuard protege las rutas privadas y guestGuard evita que un usuario con sesión vuelva al login.

Autoevaluación de la rúbrica

Esto se pidió en alguna tarea del semestre y nos sirve como checklist para saber si nos hace falta algo de la entrega

Criterio	Pts	Dónde se cumple
Capa de datos: servicios, hooks o composables, modelos tipados, entorno	15	shared/services, shared/models, environments/; Signals y rxResource
Interceptor o manejo global de errores	10	error.interceptor.ts + auth.interceptor.ts
Lista con búsqueda y paginación, y detalle por ruta	20	/explore (búsqueda, tema, paginación en servidor, tambi y en la URL) también en /quizzes/mine y detalle en /quizzes/:id
Formularios: validación, validador personalizado, edición precargada, guard de salida	30	Formulario de quiz y registro; uniqueOptions y passwordsMatch; /quizzes/:id/edit; unsavedChangesGuard
Estados de carga, error, vacío y éxito	15	<app-view-state> en las vistas con API y toasts de éxito y de error
README y PRs como flujo de trabajo	10	Readme en el repositorio, historial de comits y de PRs y este documento